



HSMA
FIND THE FUTURE IN YOU



An Introduction to Discrete Event Simulation and SimPy

Dr Daniel Chalk, Sammi Rosser

Let's Go!



Welcome



HSMA

Welcome to today's workshop!

Today we'll :

- Introduce you to a type of simulation modelling known as *Discrete Event Simulation (DES)* and the kinds of problems they can be used to tackle
- Give you a chance to interact with a DES model using a web application interface built using *StreamLit*
- Show you how these models can be structured in Python using a Python package known as *SimPy*
- Introduce the *vidigi* package for providing animated visualisations of your modelled systems

You do *not* need any prior Python programming experience for this workshop, though you should have at least coded in another language (eg R).

Quick HSMA Plug!



<https://hsma.co.uk/>

<https://www.youtube.com/@hsma>



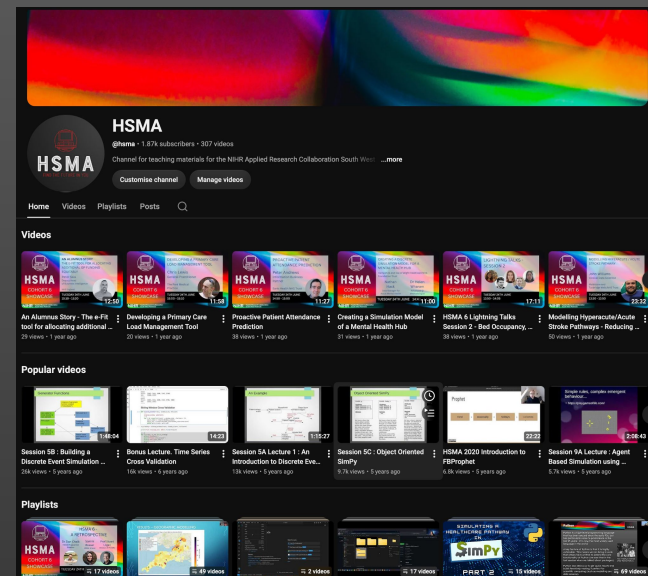
WHAT IS HSMA?

The Health Service Modelling Associates (HSMA) Programme teaches public sector analysts, clinicians and managers how to use **data science** and **operational research** techniques, as well as supporting them to apply these techniques in their organisations.

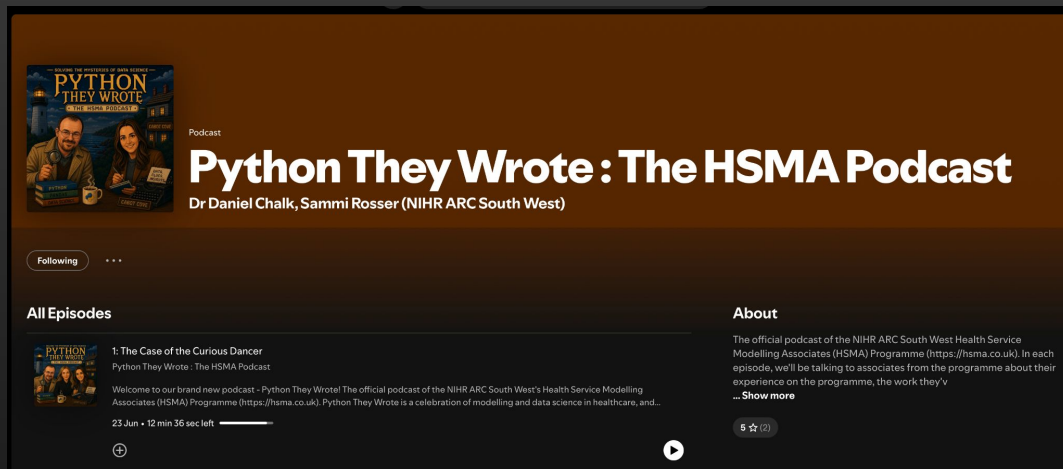
The full HSMA course experience is provided **free of charge** to people working in health, social care and policing organisations in England.

Recordings of previous course sessions, as well as all of our course materials and code, are offered **free of charge** to anyone, anywhere in the world for personal use and adaptation.

PAUSE SLIDESHOW



<https://open.spotify.com/show/033fZhKMh7wfOOvCunS9C2>



Python They Wrote: The HSMA Podcast

Dr Daniel Chalk, Sammi Rosser (NIHR ARC South West)

Following

All Episodes

1: The Case of the Curious Dancer
Python They Wrote: The HSMA Podcast

Welcome to our brand new podcast - Python They Wrote! The official podcast of the NIHR ARC South West's Health Service Modelling Associates (HSMA) Programme (<https://hsma.co.uk/>). Python They Wrote is a celebration of modelling and data science in healthcare, and...

23 Jun • 12 min 36 sec left

About

The official podcast of the NIHR ARC South West Health Service Modelling Associates (HSMA) Programme (<https://hsma.co.uk/>). In each episode, we'll be talking to associates from the programme about their experience on the programme, the work they v...

Show more

5 ☆ (2)

Quick Poll



HSMA



[menti.com/
alsh2a6b1g5r](https://menti.com/alsh2a6b1g5r)

Your responses will be used to help us evaluate and improve our training. We may also use anonymised responses for research and evaluation purposes, including in academic publications, conference presentations, reports and other research outputs. Participation is voluntary.

Queuing Problems



HSMA

The modelling approach we're going to introduce you to today is known as *Discrete Event Simulation*.

This is (by far) the most popular approach used in HSMA projects.

Discrete Event Simulation can help us to solve *queuing problems*.

But what are queuing problems?



Queuing Problems



HSMA

Health and social care systems are full of queuing problems. Some examples might include :

- Reducing waits in an Emergency Department for triage, treatment, an admission etc
- Streamlining call systems, such as 111 / GP phone systems
- Managing waits for surgical assessments and procedures
- Reducing delays in mental health assessment and planning the capacity for ongoing appointments
- Planning new services and how pathways need to be resourced to meet expected demand
- Managing hospital flows and beds
- Test results waiting to be processed and analysed



Discrete Event Simulation (DES) is a simulation modelling method that can help us tackle queuing problems.

The DES Playground



HSMA

Let's imagine you run a treatment centre that sees both trauma and non-trauma patients.

All patients who arrive are first triaged in a triage cubicle, and are then sent down one of two different pathways depending on whether they are presenting with a traumatic injury or not.

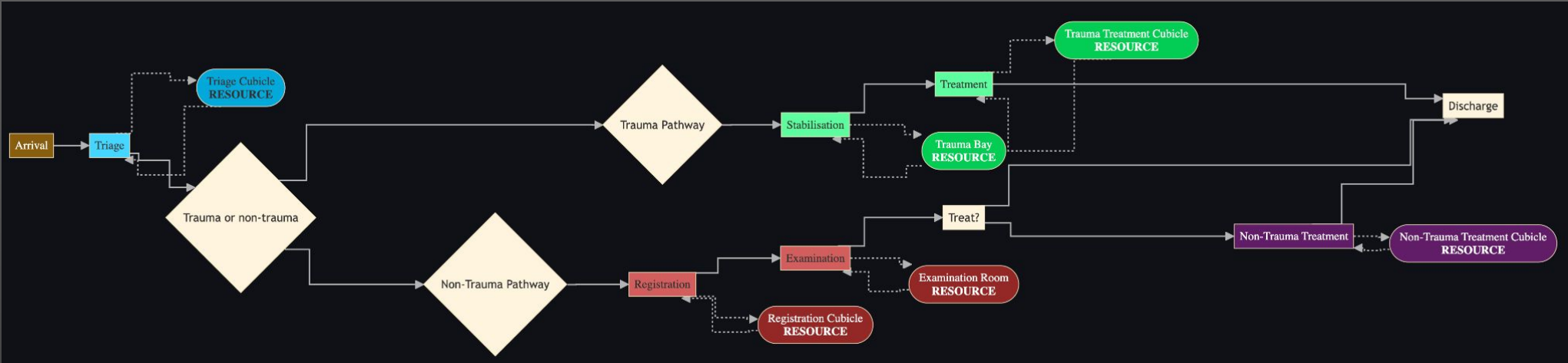
Trauma patients are first stabilised in a trauma bay, and then treated in a cubicle, before being discharged.

Non-trauma patients are first registered in a registration cubicle, and then undergo an examination in an examination room. A decision is then taken as to whether to treat them in a cubicle (separate to the trauma cubicles) or to send them home.

The DES Playground



HSMA



Patient arrivals to the centre vary across a 24 hour period. The centre tends to be busiest during the day, where there tends to be an average of around 10-20 patients arriving in an hour. During the night, things slow down and there tends to be around 2-5 patients per hour. Patient triage takes about **6 minutes** on average, and about **30%** of patients coming in have a traumatic injury.

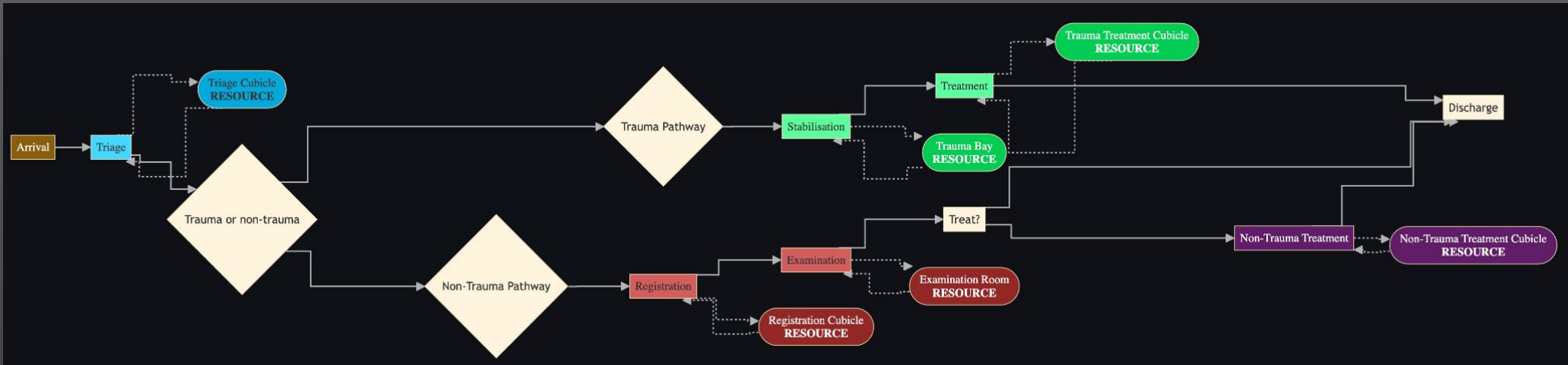
For trauma patients, it takes an average of about **90 minutes** to stabilise a patient, and about another **30 minutes** to treat them.

For non-trauma patients, registration takes about **8 minutes** and examination about **16 minutes**. About **70%** of these patients need treatment, and those patients tend to spend about **13 minutes** being treated.

The DES Playground



HSMA



In the system currently, there are :

- 4 triage cubicles
- 6 trauma bays for stabilisation
- 6 treatment cubicles for trauma patients
- 3 registration cubicles
- 3 examination rooms for non-trauma patients
- 2 treatment cubicles for non-trauma patients

The DES Playground



HSMA

We could tackle this problem using a *simulation model*.

Before we look at that though, let's talk about a couple of important concepts when we're undertaking simulation modelling.

Base Case Scenario

A representation of the real world system in our model, *as it is now*

This allows us to :

- Do a bit of a check to see if the model's providing a sufficiently decent approximation of the real world (*Validation*)
- See what's going on in the system now, identify potential issues, and possibly gain clues as to what we might want to do

"What if?" Analysis

Exploration of scenarios that make changes to the base case scenario to see what might happen if those changes were made in the real world.

This allows us to :

- Get a sense of the kinds of things that might help
- Assess what the potential impact of proposed changes might be

DES Playground



HSMA

We said we could have built a simulation model to tackle this. And we have! Specifically we've built a *Discrete Event Simulation* - the method you'll be learning in this workshop.

Let's introduce you to the *DES Playground*.

<https://playground.hsma.co.uk/>



HSMA
FIND THE FUTURE IN YOU

The DES Playground

- Introduction
- 1 Simulating Arrivals
- 2 Using A Simple Resource
- 3 Adding an Optional Step
- 4 The Full Model
- 5 The Code
- 6 Find Out More

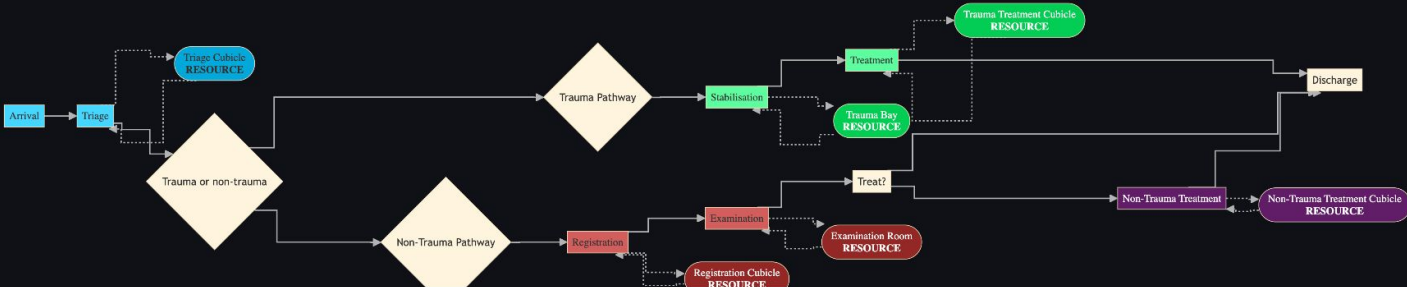
Welcome to the Discrete Event Simulation Playground! 🙌

This is a discrete event simulation playground based on the Monks et al (2022), which is itself an implementation of the Treatment Centre Model from Nelson (2013).

By working through the pages on the left in order, you will

- see how a discrete event simulation builds from simple beginnings up to the point of being able to model a complex system
- understand the impact of variability and randomness on systems
- have a go at changing parameters to find the best configuration that balances the average use of resources (like treatment bays or nurses) and the average time a patient will wait at each step of the process.

The treatment centre we want to build a model of looks like this:



```
graph LR
    Arrival[Arrival] --> Triage[Triage]
    Triage --> TraumaOrNonTrauma{Trauma or non-trauma}
    TraumaOrNonTrauma --> TraumaPathway[Trauma Pathway]
    TraumaPathway --> TriageCubicle[Triage Cubicle RESOURCE]
    TriageCubicle --> Stabilisation[Stabilisation]
    Stabilisation --> Treatment[Treatment]
    Treatment --> TraumaTreatmentCubicle[Trauma Treatment Cubicle RESOURCE]
    TraumaTreatmentCubicle --> Discharge[Discharge]
    TraumaPathway --> TraumaBay[Trauma Bay RESOURCE]
    TraumaBay --> Discharge
    TraumaOrNonTrauma --> NonTraumaPathway[Non-Trauma Pathway]
    NonTraumaPathway --> Registration[Registration]
    Registration --> RegistrationCubicle[Registration Cubicle RESOURCE]
    RegistrationCubicle --> Examination[Examination]
    Examination --> ExaminationRoom[Examination Room RESOURCE]
    ExaminationRoom --> NonTraumaTreatment[Non-Trauma Treatment]
    NonTraumaTreatment --> NonTraumaTreatmentCubicle[Non-Trauma Treatment Cubicle RESOURCE]
    NonTraumaTreatmentCubicle --> Discharge
```

References

1. Monks, I., Harper, A., Anagnostou, A., Allen, M., Taylor, S. (2022) Open Science for Computer Simulation; [Repository Link](#)
2. Nelson, B.L. (2013). *Foundations and methods of stochastic simulation*. Springer.
3. <https://health-data-science-or.github.io/simpy-streamlit-tutorial/>

The DES Playground



HSMA

The DES Playground is an example of a *web application* (built using a package known as *StreamLit*), that provides a user friendly interface to a *Discrete Event Simulation* running underneath (built using a package known as *SimPy*).

Note

The model and web app were developed by Sammi, and the model's based on a Treatment Centre Simulation developed by some of our colleagues (Tom Monks et al) :

<https://github.com/TomMonks/treatment-centre-sim/tree/main>, which is itself based on an example in a 2013 stochastic simulation textbook by Barry Nelson ("Foundations and Methods of Stochastic Simulation: A First Course")

The DES Playground



HSMA

The app talks you through building up components of a model like this bit by bit. But for now, we're going to skip ahead to the full model - click on "The Full Model" in the sidebar, and then on the "Playground" tab.

Here, you can see some of the *parameters* of the model - things we can feed in and change to see what might happen if we pull certain levers. This allows us to conduct "What if?" Analysis.

Triage

Number of Triage Cubicles

1 10

4

Probability that a new arrival is a trauma patient

0.00 1.00

0.30

Trauma Pathway

Number of Trauma Bays for Stabilisation

1 10

6

Number of Treatment Cubicles for Trauma

1 10

6

Non-Trauma Pathway

Number of Registration Cubicles

1 10

3

Number of Examination Rooms for non-trauma patients

1 10

3

Non-Trauma Treatment

Number of Treatment Cubicles for Non-Trauma

1 10

2

Probability that a non-trauma patient will need treatment

0.00 1.00

0.70

Total rooms in use is 24

Advanced Parameters

Run simulation

The DES Playground



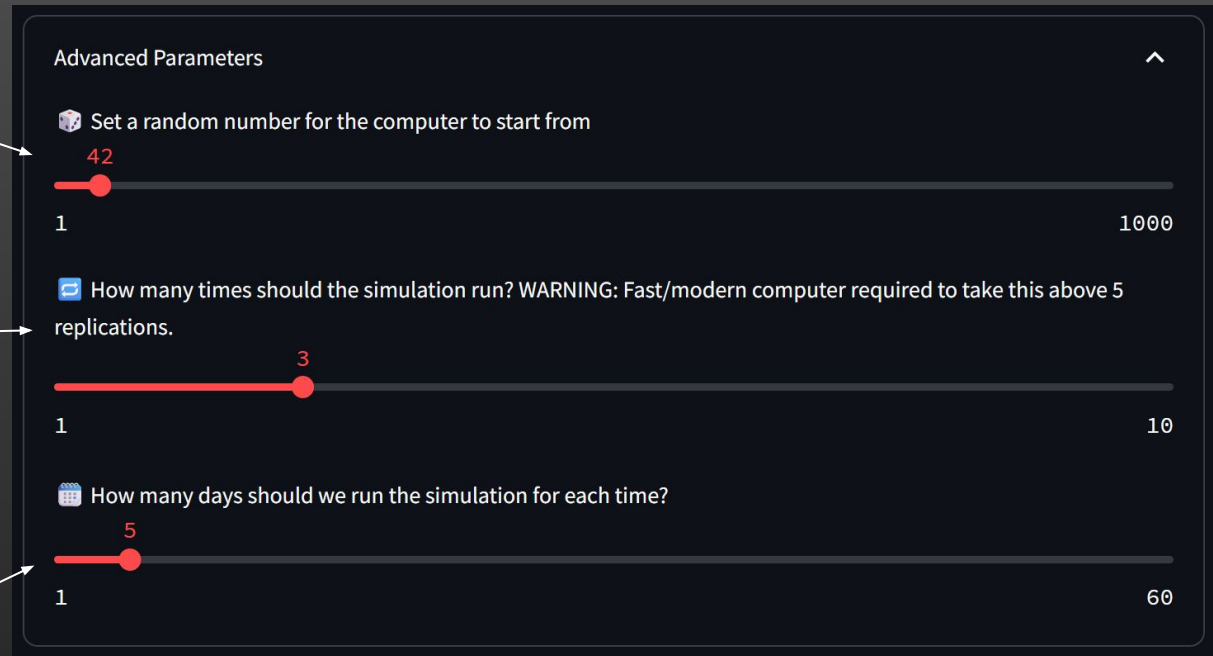
HSMA

There's also a drop down list of Advanced Parameters to allow us to access parameters relating to the simulation itself.

This is just a way of setting something known as the “Random Seed”. We'll come back to this later

How many replications of the model we want to run (times we want to run it). We'll come back to that later too.

How many “virtual days” should we simulate in a run of the simulation?



Advanced Parameters

 Set a random number for the computer to start from

42

1 1000

 How many times should the simulation run? WARNING: Fast/modern computer required to take this above 5 replications.

3

1 10

 How many days should we run the simulation for each time?

5

1 60

The DES Playground



HSMA

When we click the “Run Simulation” button, the model underneath runs with the parameter values we’ve specified. Once it’s finished, it’ll display a variety of outputs. Let’s have a look at some of these now.

Run simulation

The DES Playground

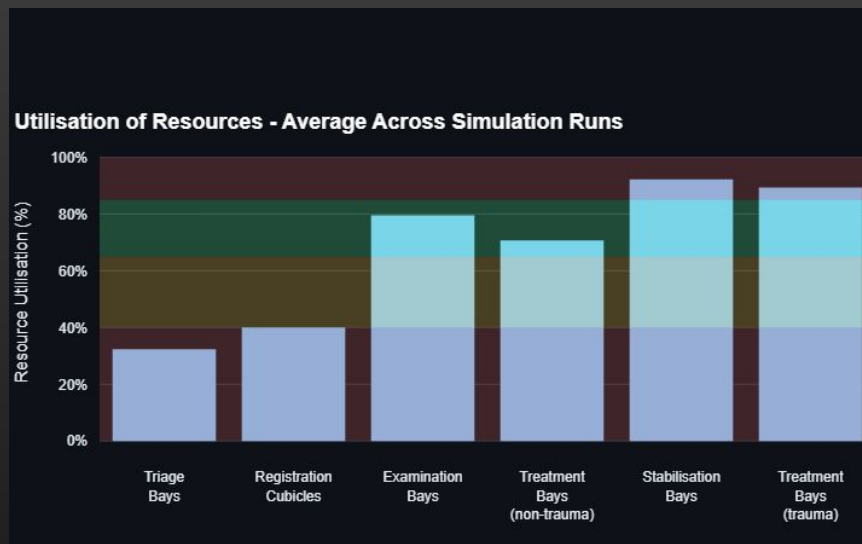


HSMA

Utilisation indicates how often a resource was being used during the simulation, and is measured as a percentage.

Waiting times are often the key metrics we're looking at in a DES model.

We might also be interested in how long patients spend in the system in total. Or the % that breach a target (eg 4 hour ED target).



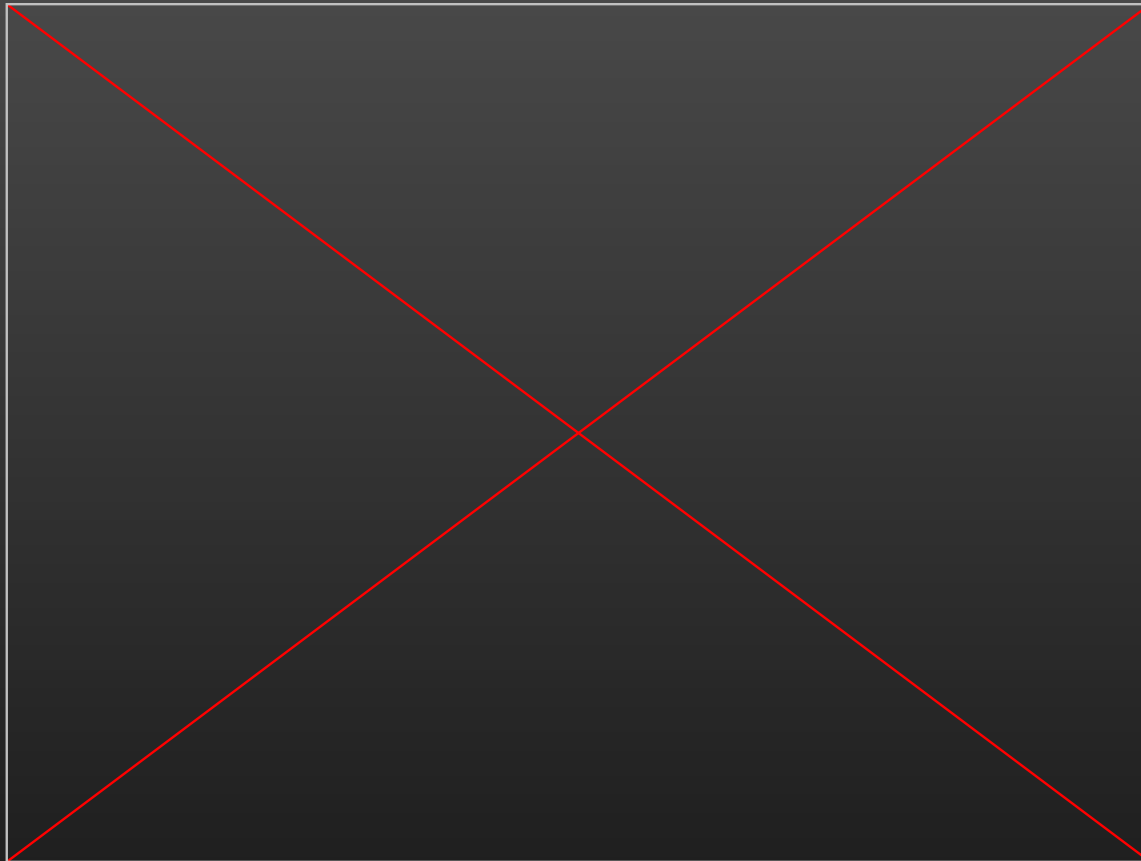
The DES Playground



HSMA

In this app, we can also look at an animation of what happened during a simulation run, so we can see our patients moving through our simulated pathway. This can be useful to get a feel for what's happening.

Here, we use a package known as *vidigi* (authored by our very own Sammi Rosser) to do this.



The DES Playground



HSMA

This app also allows us to compare different “what if?” scenarios, so we can see whether our changes improve things (or make things worse!).

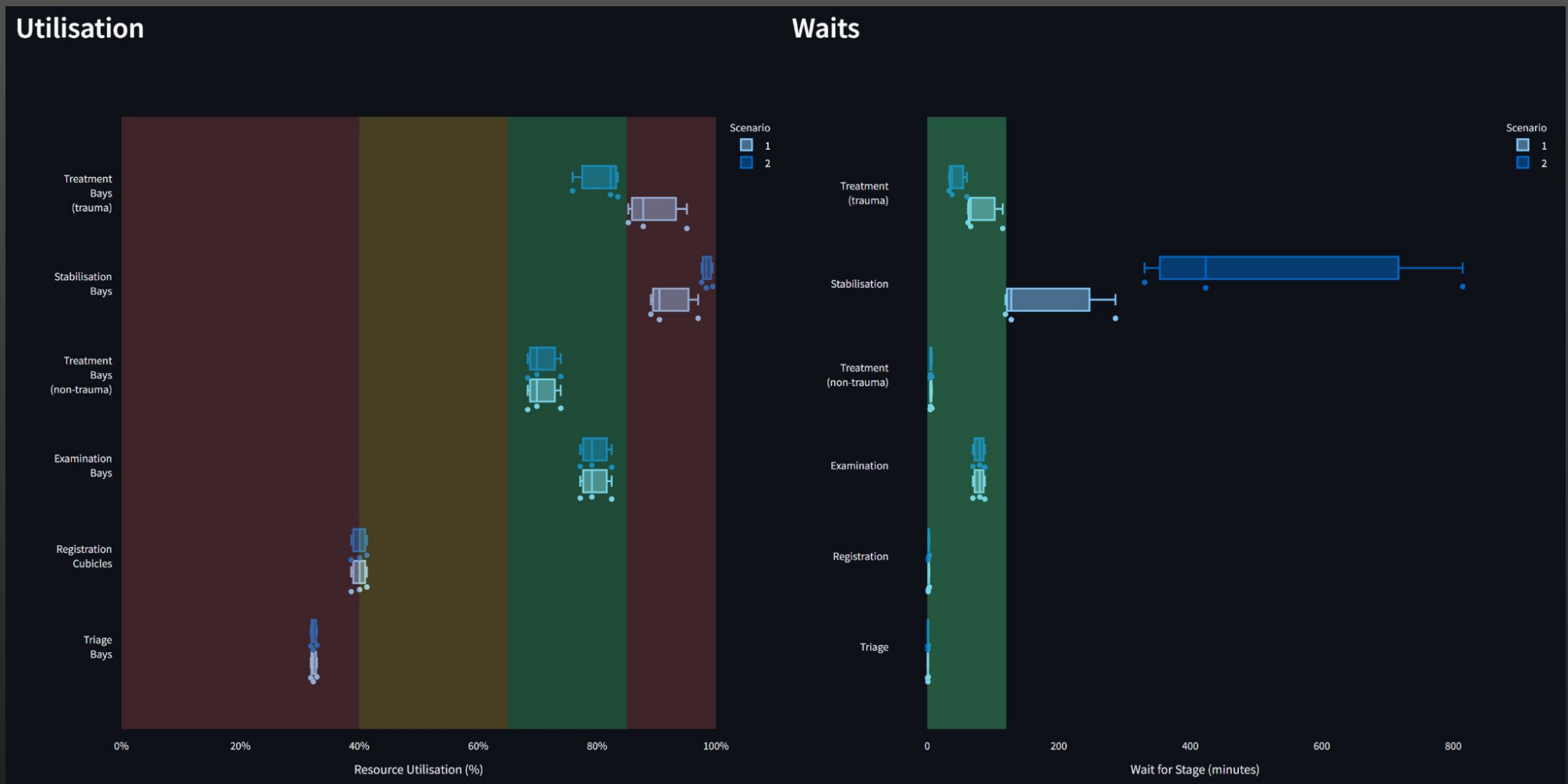


The DES Playground



HSMA

We can also look at a more advanced view to understand the breadth of the data from across the replications. Don't worry too much about that right now.



The DES Playground



HSMA

And we can get some raw numbers out so we can start collating results to help inform decisions....

This displays the median value for each metric across all model runs per scenario.

Metric	Scenario 1	Scenario 2
00 Arrivals	1560	1560
01A Triage Wait	1.0 minutes	1.0 minutes
01B Triage Util	32.3%	32.3%
02A Registration Wait	1.6 minutes	1.6 minutes
02B Registration Util	40.1%	40.1%
03A Examination Wait	79.9 minutes	79.9 minutes
03B Examination Util	79.1%	79.1%
04A Treatment Wait(Non Trauma)	5.0 minutes	5.0 minutes
04B Treatment Util(Non Trauma)	69.9%	69.9%
05 Total Time(Non-Trauma)	127.5 minutes	127.5 minutes
06A Trauma Wait	127.8 minutes	423.4 minutes
06B Trauma Util	90.5%	98.4%
07A Treatment Wait(Trauma)	65.9 minutes	37.8 minutes
07B Treatment Util(Trauma)	87.8%	82.3%
08 Total Time(Trauma)	370.9 minutes	622.6 minutes
09 Throughput	1484	1439

...Though bear in mind, these models aren't crystal balls. They're not designed to predict exactly what will happen, but rather give you a better sense of the direction of travel. So be careful how you interpret results.

Exercise



HSMA

You will now split into small groups for **25 minutes**. You're going to use the DES Playground to study the base case scenario, and then perform some "what if?" analysis, to answer the following questions :

1. Which resources are being over-utilised in the system? (in use more than 85% of the time?)
2. Which resources are being under-utilised in the system (in use less than 65% of the time?)
3. Are any bottlenecks in the system (long delays), and if so, where are they? And how long are those waits?
4. Your department has managed to secure some additional space and funding to locate some additional resource. Specifically, you can increase *two* of the existing cubicles / rooms. Which two would you pick?

Once you've done that, you'll also answer the following question :

5. Imagine you've been given free reign to resource the department centre however you like. What is the minimum configuration of resourcing you think you need across the system in order to ensure that a) as many resources remain as well utilised as possible, and b) wait times are as low as possible?

After the exercise, take a **10 minute** break before we rejoin.

Discrete Event Simulation

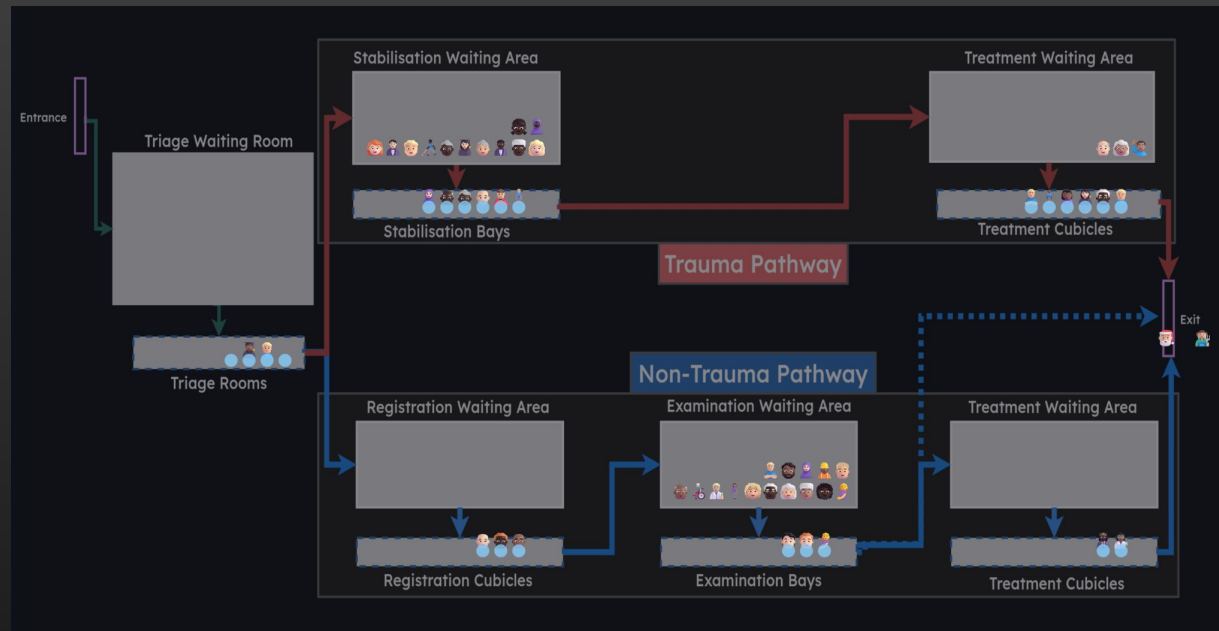


HSMA

A Discrete Event Simulation uses computer simulation to create an *in-silico* (digital) version of a real world system. Specifically it models things flowing through this system, queuing up waiting for *discrete sequential processes* (individual things happening in some order) that use some *resources*.

We can run these models and assess what might happen to things like waiting times to help inform decisions about changes to the system (“what if?” analysis).

DES models are made up of a number of components. Let’s look at them now.



DES Components



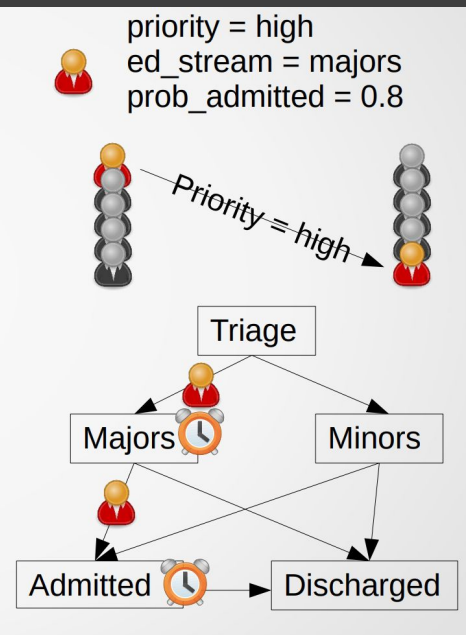
HSMA

Entities

The things flowing through the system (e.g. patients, test results, telephone calls). Can have more than one type.



Entities are the things flowing through our modelled system. They might represent patients, test results, callers on the phone - they are the things to which things will happen in our system. We might also have more than one type of entity in our model.



Each entity can carry certain information with them (e.g. age, condition, severity etc), which may help determine its journey through the system - rarely are systems entirely linear and likely branch off (or even loop back on themselves)

DES Components



HSMA

Generators

How entities arrive into the model. Can have multiple.

Inter-arrival times (IAT) determine arrival frequencies.



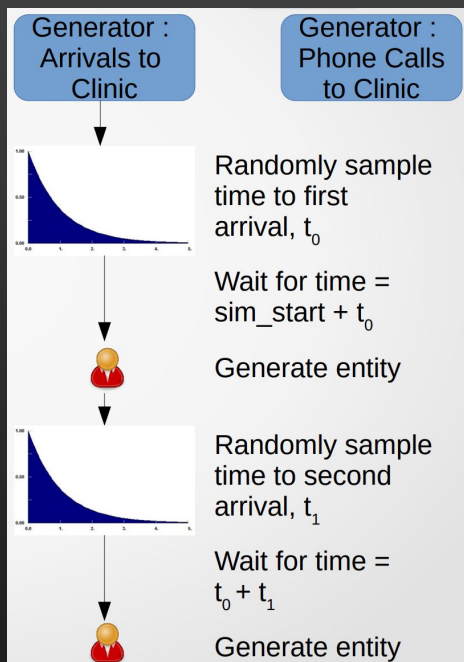
Generators create new entities in a DES - they bring our entities into being. They represent the ways in which our entities can enter the system we're modelling (e.g. arriving at reception, transferred to the ward etc).

We may have more than one generator if we are :

- Generating different types of entity
- Generating entities who have different patterns of arrival (eg patients arriving more frequently via self-presentation than ambulance etc)

The rate at which entities are generated is determined by an *inter-arrival time (IAT)*, which represents the time between an entity being generated and the next one being generated.

Inter-arrival times may be fixed, but are typically sampled (drawn) stochastically (randomly) from a *probability distribution* (usually an *Exponential Distribution*) to capture variability. More than one distribution may be used for the same generator (e.g. for different times of the day, day of week etc). We'll talk more about distributions later.



DES Components



HSMA

Activities

The things that happen to our entities.
Activity times determine how long an activity takes for a given entity.

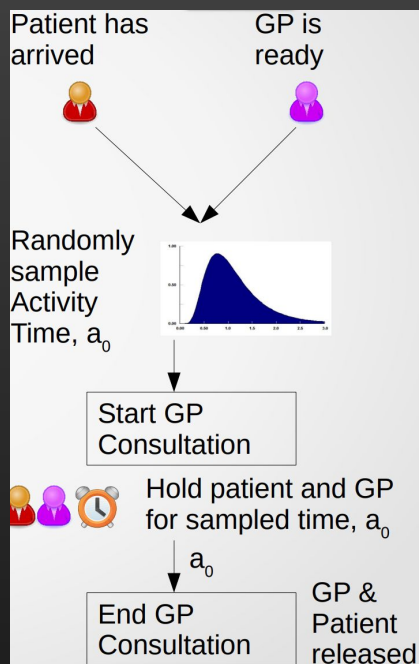


Each *activity* in a DES describes something that happens to our entities which takes an amount of time and (usually) some resources. For an activity to take place, it needs :

- An entity to be ready
- Any necessary resources to be ready

Once the above conditions have been met, the activity begins. The entity, and the resource(s) are then locked in place for an amount of time – the *Activity Time*. The resource(s) cannot be used elsewhere until the activity time has passed, at which point the entity moves on and the resource(s) become available again.

Activity times may be fixed, but are typically sampled stochastically from a distribution (usually a Log Normal distribution).



DES Components



HSMA

Queues

Where entities wait if resources are not ready. Can configure different queuing policies (e.g. FIFO, priority)



First in, First Out
(FIFO)



Priority-based



Each activity in a Discrete Event Simulation has an associated queue. The queue holds entities whilst they wait for the resources for an activity to become available for them.

Each queue has a *queuing policy*. This determines the order in which entities are released from the queue into the activity. The two most common queuing policies are:

- *First In First Out (FIFO)*: entities are seen in the order they arrive. This is the default.
- *Priority-based*: entities are seen according to some priority attribute. Ties often resolved using FIFO

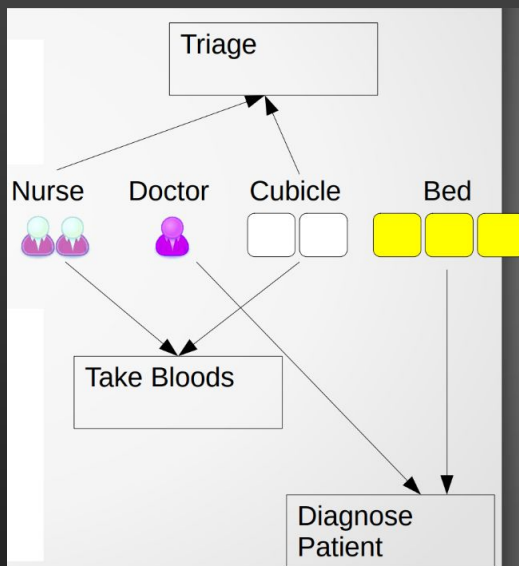
DES Components



HSMA

Resources

The things that an activity needs to take place (staff, stuff). An activity starts when entity and resources are ready, and both locked in place for activity time.



Resources are needed to undertake activities. An activity may require just a single resource, more than one resource of the same type, or multiple resources of different types.

An activity may not require a resource at all, but think carefully to ensure that it really is either :

- “Resourceless”
- there is no constraint on the resource compared to demand (and so doesn’t need to be modelled, as the resource will always be available)
- It is not relevant to the modelling question, and your “what if?” scenarios won’t affect the resource levels

Resources can include “staff” (e.g. doctors, nurses, admin staff etc) and “stuff” (beds, test equipment, medication etc)

Resources can (and often are) shared across a system, so may be required for more than one activity. Therefore, a resource drain in one part of the system can affect another.

All required resources are needed for an activity to take place.

DES Components



HSMA

Sinks

How entities leave the model. Can have multiple.

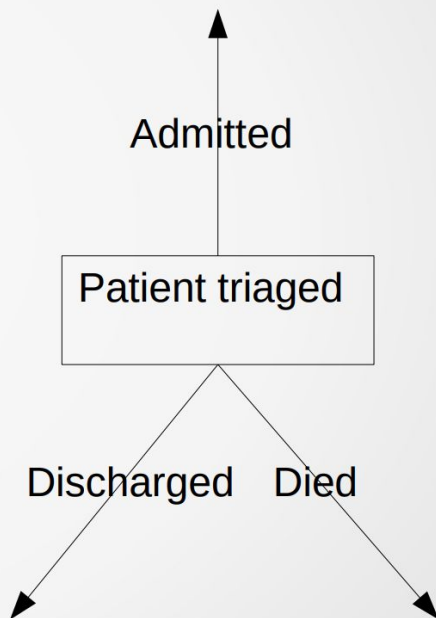


Sinks are how entities leave the system being modelled. Sinks might include :

- an entity physically leaving a system (e.g. discharge from hospital)
- an entity no longer existing (e.g. death, use of sample, end of telephone call)
- an entity no longer needing to access activities that we're interested in (e.g. they leave the bit of the real world system that we're modelling)

The most important thing to remember about a sink is that it doesn't necessarily represent an entity leaving the real world system entirely.

For example, the *scope* of your model may only cover the Emergency Department. Many patients will be admitted to the hospital, but if you're not modelling that system, then they simply leave your model via a sink.



Branching Paths

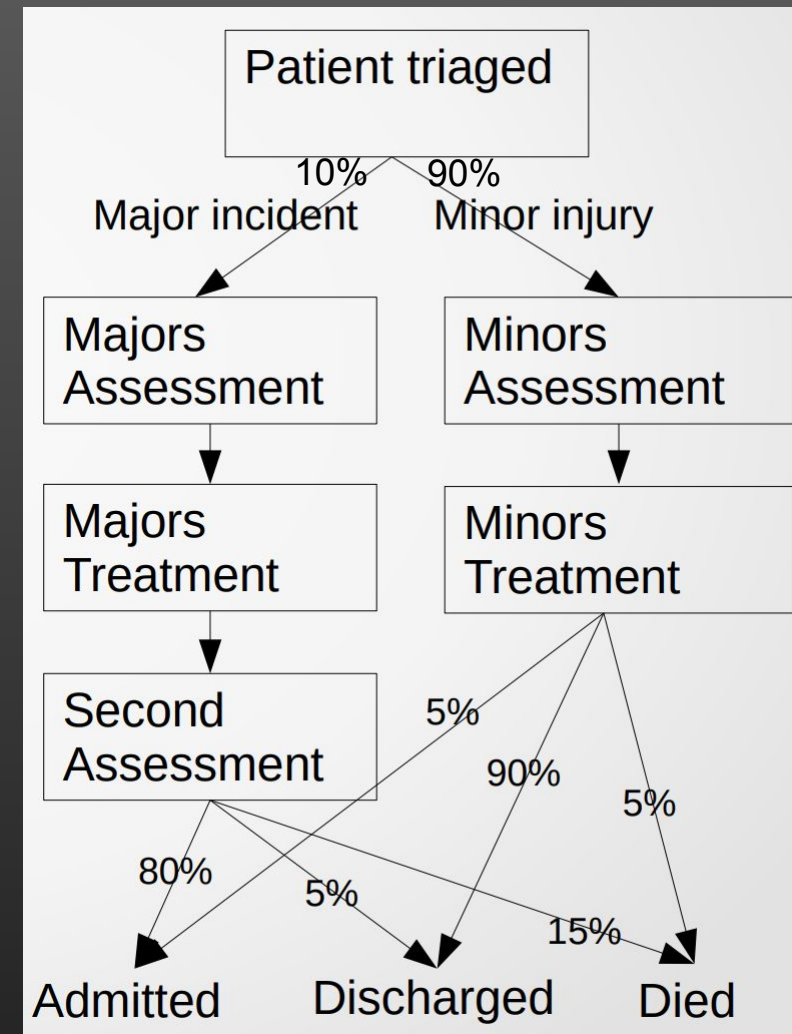


HSMA

Real world systems (and the models of those systems) are rarely linear. Often, different things will happen to different entities. In a Discrete Event Simulation, this means different entities flowing to different activities, or different sinks.

We might differentiate based on :

- an attribute of the patient (e.g. patients with more severe conditions through a different set of activities)
- probability (e.g. we know that approx 60% of these patients end up being admitted, so we'll randomly select for them to be admitted 60% of the time)
- time (e.g. after a certain time of day, entities flow through a different set of activities)



DES Components Summary



HSMA

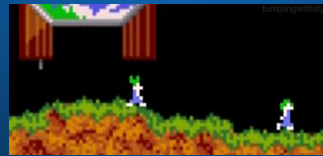
Entities

The things flowing through the system (e.g. patients, test results, telephone calls). Can have more than one type.



Generators

How entities arrive into the model. Can have multiple. *Inter-arrival times (IAT)* determine arrival frequencies.



Sinks

How entities leave the model. Can have multiple.



Activities

The things that happen to our entities. *Activity times* determine how long an activity takes for a given entity.



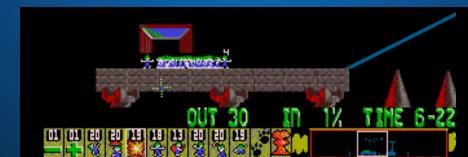
Resources

The things that an activity needs to take place (staff, stuff). An activity starts when entity and resources are ready, and both locked in place for activity time.



Queues

Where entities wait if resources are not ready. Can configure different queuing policies (e.g. FIFO, priority)

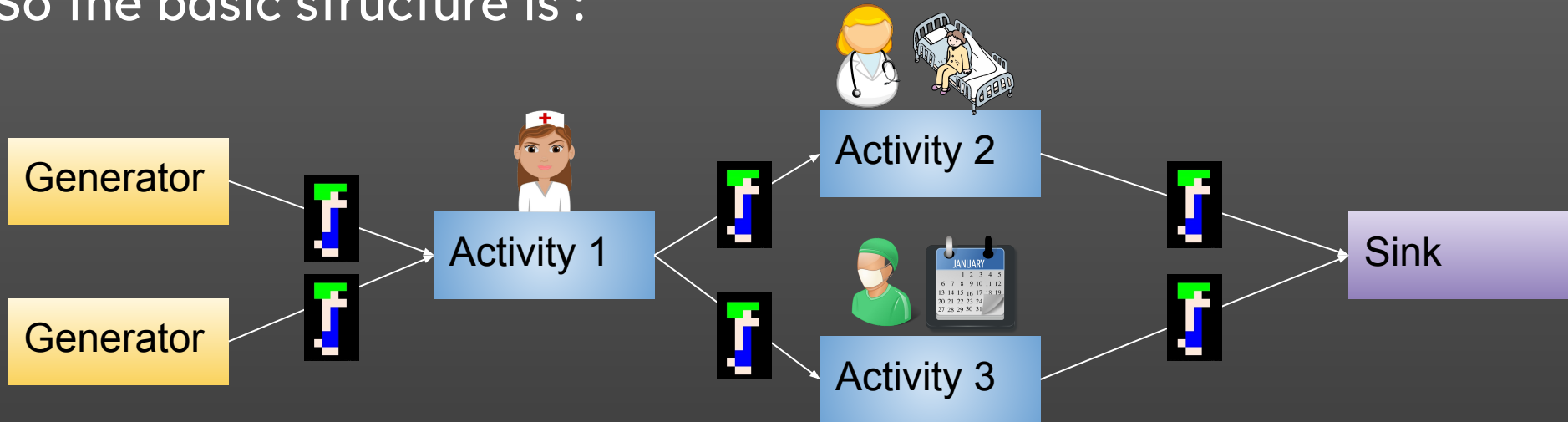


DES Components Summary



HSMA

So the basic structure is :



Disclaimer : Lemmings used for illustrative purposes only. We in no way take any responsibility should you choose to describe patients in your pathways as Lemmings, or install a lava pit at the exit to your service.

Has Dan got time for a pasty?



Consider the following scenario. There's a local wellbeing clinic that opens at 8am. People are seen as walk-ins without appointments, and tend to turn up on average every 5 minutes throughout the day, and spend on average 5 minutes with the wellbeing advisor.

Dan's considering visiting at around 1pm, but is worried about whether he'll have time to grab a pasty on the way home before they sell out. He reckons as long as he's not in the clinic more than about 20 minutes he should be ok. (And no - he can't grab it before. It'll go cold.)

Given the importance of this decision, he's decided to create a model of the clinic (and his planned arrival) to try to work out if he has time to go to the clinic and get a pasty.

He first decides to model the clinic by just assuming that the IAT and activity times are fixed (at 5 minutes each). Let's have a look at what he concludes.

Has Dan got time for a pasty?



How long does Dan wait?

Summary statistics

Mean wait (min)

0.0

Median wait (min)

0.0

90th percentile (min)

0.0

Looks like Dan won't wait at all. Everyone happy?

Has Dan got time for a pasty?



Whilst Dan was trying to decide whether he'd go for a traditional steak or a cheese and onion, he starts wondering how reliable his model is.

The data clearly shows that people arrive on average every 5 minutes, and spend on average 5 minutes with the advisor. He can see that there's a bit of variability around these numbers, but not much (looks like the standard deviation around the activity time is about 2 minutes). It should all come out in the wash surely?

Just to be on the safe side (there is a pasty on the line here), Dan decides to model the variability in the real world system. Let's have a look at what he concludes now.

As there's randomness now, he runs the model 500 times and takes average results over these runs.

Has Dan got time for a pasty?



How long does Dan wait?

Summary statistics

Mean wait (min)

32.6

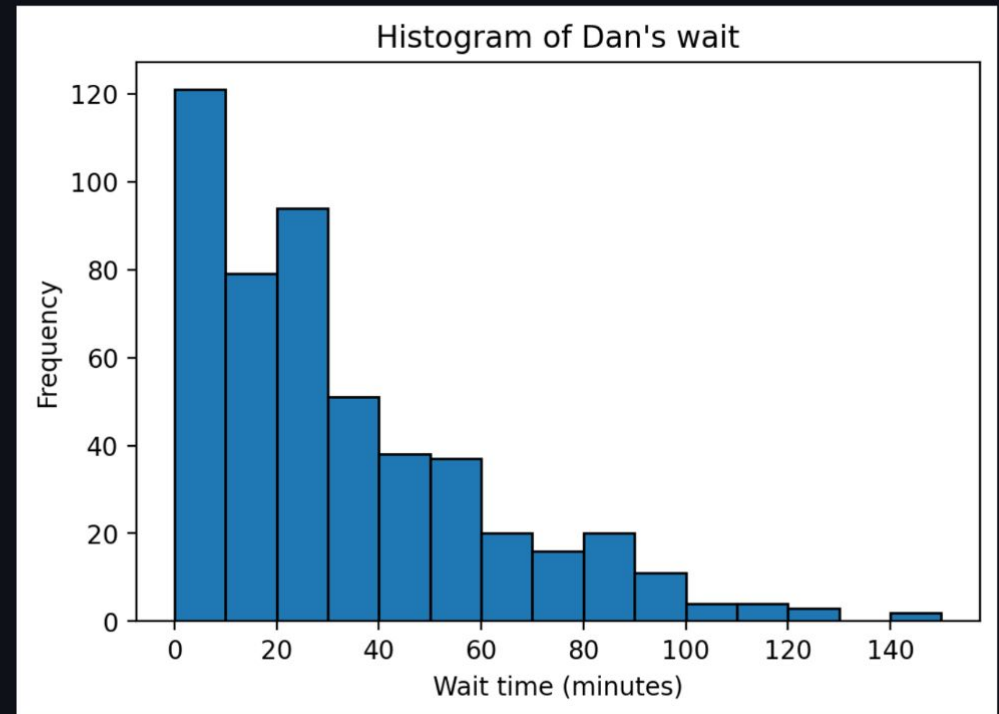
Median wait (min)

24.2

90th percentile (min)

76.1

Distribution of Dan's waiting time



Dan likely won't have time for a pasty!!!! If he's really unlucky, he might even be waiting so long they don't even have any sausage rolls left (which always sell out last)

What's happened? Let's work through an example.

Has Dan got time for a pasty?



Here's some data you've found for the first 10 patients from a day of this clinic running.

Patient 1 arrives 1 min after the clinic opens, and spends 3.2 mins with the advisor.

Patient 2 arrives 2 mins later, and spends 4.1 mins with the advisor.

Patient 3 arrives 3 mins later, and spends 4.5 mins with the advisor.

Patient 4 arrives 4 mins later, and spends 5 mins with the advisor.

Patient 5 arrives 5 mins later, and spends 5.4 mins with the advisor.

Patient 6 arrives 6 mins later, and spends 5.9 mins with the advisor.

Patient 7 arrives 7 mins later, and spends 6.1 mins with the advisor.

Patient 8 arrives 8 mins later, and spends 4.8 mins with the advisor.

Patient 9 arrives 9 mins later, and spends 3.7 mins with the advisor.

Patient 10 arrives 5 mins later, and spends 7.3 mins with the advisor.

Mean interarrival time for 1st 10 patients = 5 minutes

Mean activity time for 1st 10 patients = 5 minutes

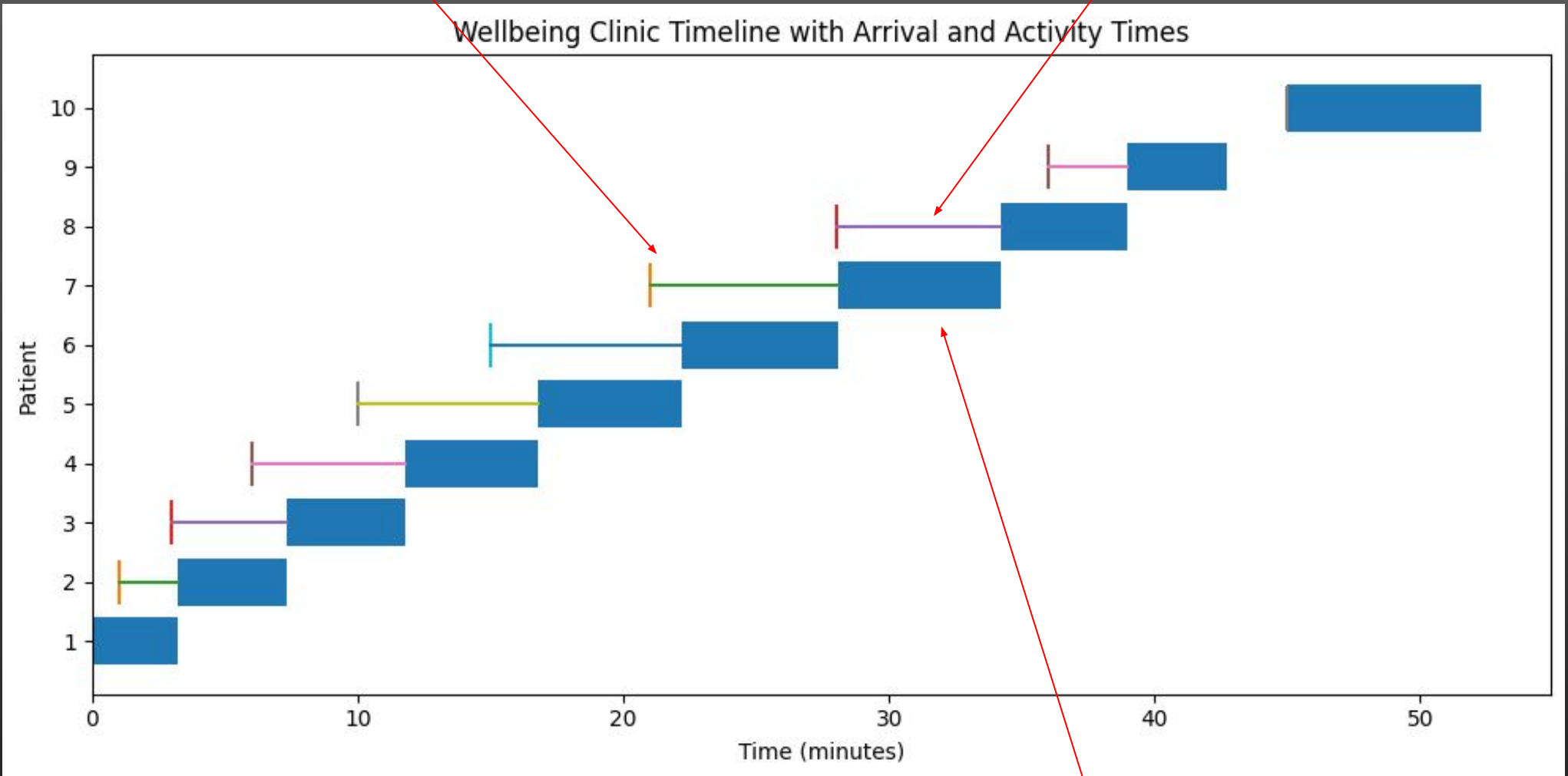
Let's visualise this to see what's happening more clearly.

Has Dan got time for a pasty?



Arrival times (vertical lines)

Patient waits (horizontal lines)



Times with advisor
(bars)

Variability



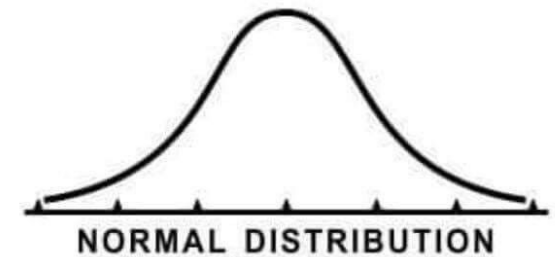
HSMA

Patients may well arrive, on average, every 5 minutes. But this doesn't mean that every patient turns up every 5 minutes on the dot.

Similarly, patients may well spend an average of 5 minutes with the advisor. But that doesn't mean the advisor has a stopwatch and stops advising them once the five minutes are up, or drags it out if they finish up quicker.

Real life is full of variability. And variability guarantees waits. If you don't model variability that exists, you're not modelling what's really happening.

To capture variability in our models, we can use *probability distributions*.

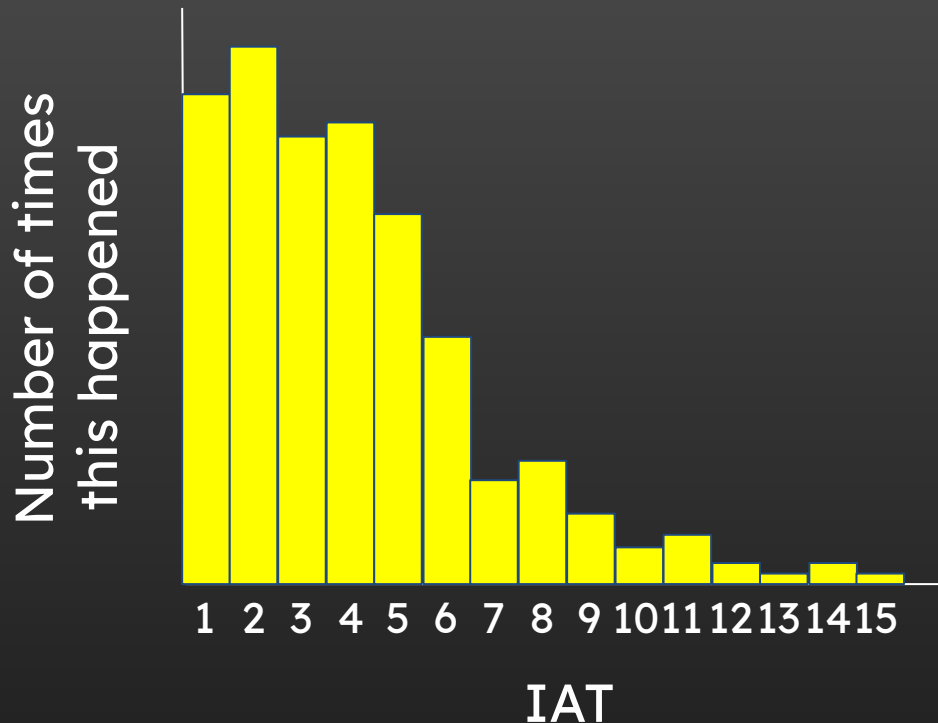


Probability Distributions



HSMA

Let's imagine that we run the wellbeing clinic and we record what time people arrive. If we were to subtract the time difference between each patient arriving and the previous patient arriving, we'd have the inter-arrival time for each patient. We could plot those on a histogram showing the number of times the inter-arrival time was 1 minute, 2 minutes etc.



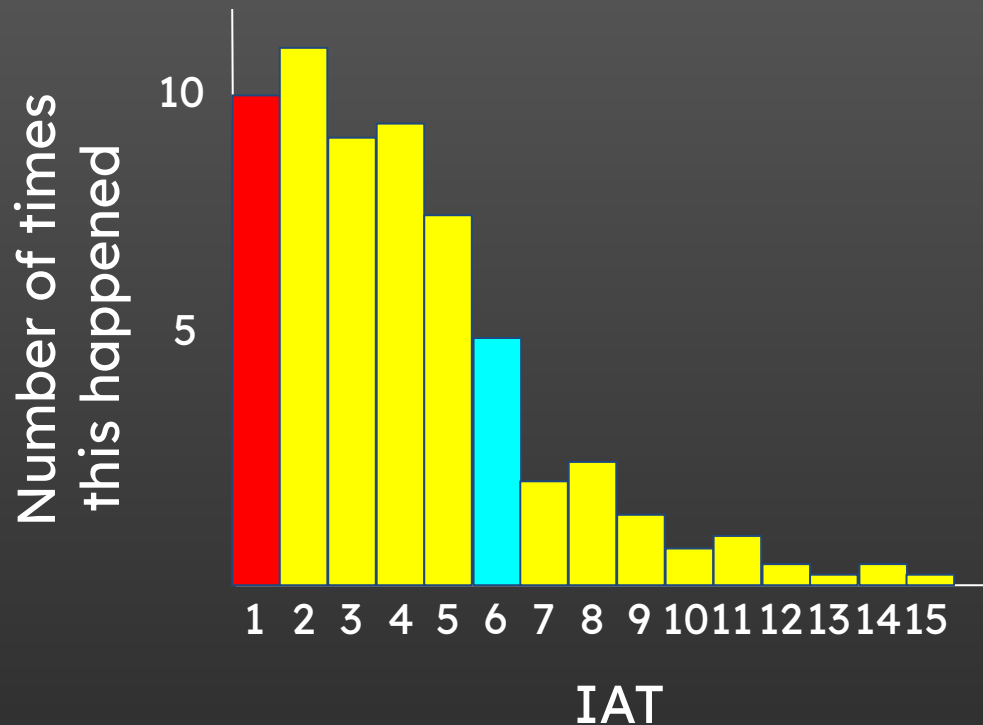
This chart basically tells us that most people arrived fairly close to each other, but there were a few times when we waited a while between arrivals.

Probability Distributions



HSMA

We could use this chart as a way of determining the probability that the next patient through the door will be in x minutes.



In this example, it's twice as likely that the next patient will arrive in 1 minute (red) than they will arrive in 6 minutes (blue).

We could use the above as a *Probability Distribution* in our model. In other words, when the generator needs to know how long to wait before generating the next patient, it will randomly (*stochastically*) pick (*sample*) from the times above, but the chance that it picks the numbers with higher bars will be greater.

Probability Distributions



HSMA

The key problem with using the real data directly like this is that we run the risk of *overfitting* to our *sample*.

What do we mean by this?

Any data we collect (or extract from our systems) represents a *sample* of all the data that ever was or could ever be for what we're looking at. It doesn't contain all of these values, because that's not (usually) possible. We could use a bigger sample, but this :

- Isn't always possible
- Might be problematic (older data may not be representative)
- Will still be a sample

Our sample won't be the same as all the theoretical data. It may not differ much, or it might differ hugely - we don't know. It's highly likely our sample doesn't contain values that might happen in the future.

If we base our arrival patterns on the sample directly, there's a danger we *overfit* - this is where we have tailored things too much to the sample, and lose *generalisability*.



Probability Distributions

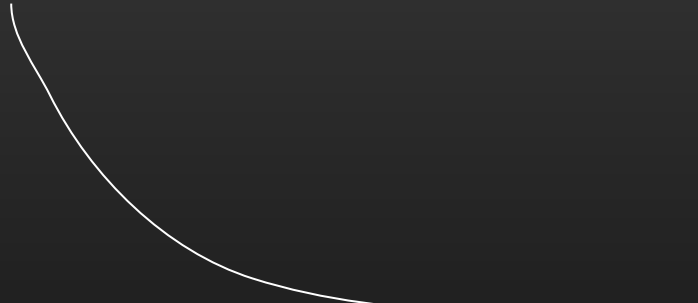


HSMA

So what do we do? We typically use a *Named Distribution*. This is a distribution that has a known shape and characteristics, and we provide it some data to make it look like our data.

To pick a named distribution, we can either use statistical methods to find the one that most closely fits (perhaps using a dedicated package such as distfit - <https://github.com/erdogant/distfit>), or use common standards.

For example, in many cases, inter-arrival times can be captured well using an *Exponential Distribution*. This is because an exponential distribution can model the distance between events in something known as a *Poisson Process* - basically, events which occur continuously and independently at a constant average rate. Like people turning up at the door over the course of the day / a period of time. An exponential distribution is *right-skewed* (it has a long “tail” that stretches off to the right), which means that most values are small, but large values are possible (albeit rare). It looks a bit like this :

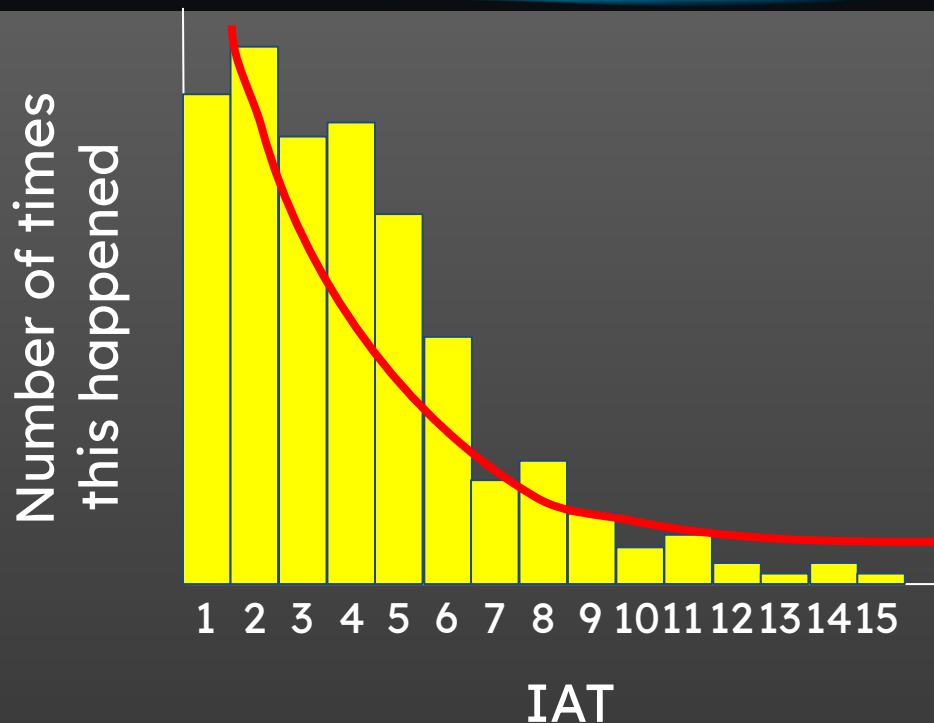
A simple line graph on a dark background. The curve starts at a high point on the left and slopes downwards to the right, forming a long tail. The curve is white and smooth.

Look familiar?

Probability Distributions



HSMA



It likely won't be a perfect match to our sample. But what we're doing is making an assumption based on what we can see - that if we had access to all data that ever was and ever could be, it would follow the exponential distribution.

To define an exponential distribution for our data, all we need to give it is one parameter value - known as *Lambda* (λ)

λ here is 1 divided by something known as *Mu* (μ). Which sounds complicated, but *Mu* is basically just the average (mean) of your data. So all we need to define an exponential distribution of inter-arrival times to our service is to know what the average is.



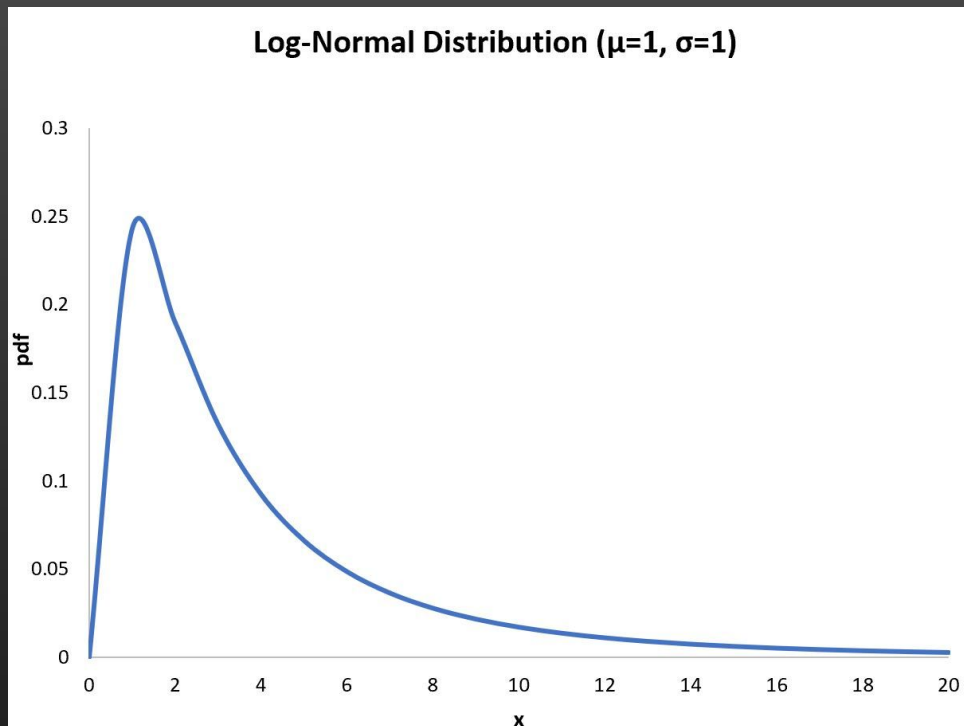
Probability Distributions



HSMA

We also use probability distributions to model activity times (the time an entity spends in an activity - eg seeing the advisor). We tend to use something known as the *Lognormal* distribution for activity times, which is also a *right-skewed* distribution.

In other words, it says that most activity times will be similar, but some will be longer, and an occasional few will be **much** longer.



For a Lognormal distribution we need two parameters - Mu (μ) and Sigma (σ).

How we get these is complicated, but all you need to know is that we can get them if we know how long an activity takes on average (mean) and how much it tends to vary (standard deviation). There are packages that convert these to Mu and Sigma for us.

Model Replications



HSMA

In a stochastic (random) model, each time we run the simulation, something different will happen because different numbers will be drawn from our distributions. This is a good thing.

However, this also means that we shouldn't just run our model once and make decisions based on that. We could have had a run where the service was particularly busy, or particularly quiet.

Instead, we run our model lots of times, and take average results over these runs.

Each run of the model is known as a *replication*. Think of it like replaying the same day / week / month (whatever your time period is) over and over in parallel universes.



We tend to describe a batch of replications / runs as a *trial*

Distributions Summary



HSMA

- We tend to use Exponential distributions to sample inter-arrival times into our system. This just needs us to know the average (mean) inter-arrival time in the real world system
- We tend to use Lognormal distributions to sample activity times in our system. This just needs us to know the average (mean) and Standard Deviation of our real activity times
- We can have different distributions in our model that kick in at different times of day, different seasons etc, or which are used for different patients etc.
- When we use distributions, our model becomes stochastic (random). To interpret results, we need to run the model multiple times (replications) and derive average results over the batch of replications (trial)

Random Seeds



HSMA

We've talked a lot about sampling from distributions randomly. But how does a computer do that?

In reality, the way in which a computer takes random actions, isn't actually random, it's using *pseudo-random number generator* to generate a random number. The numbers generated aren't truly random, but look random enough to work (for most things).

Let's look at an example of how this might work for sampling from a distribution.

Random Seeds



HSMA

Pseudo-random number generators generate numbers between 0 and 1. They take a number, perform some mathematical operations on it, then use the result (or part of it) as the input to generate the next number.

As a very simplified example, imagine the rule is multiply the starting number by 7, then add 0.3, and only keep the decimal part. Then use that result as the input to generate the next number.



E.g.

$$(0.4 \times 7 + 0.3) = 3.1 \rightarrow \underline{0.1}$$

$$(0.1 \times 7 + 0.3) = 1.0 \rightarrow \underline{0.0}$$

$$(0.0 \times 7 + 0.3) = 0.3 \rightarrow \underline{0.3}$$

$$(0.3 \times 7 + 0.3) = 2.4 \rightarrow \underline{0.4}$$

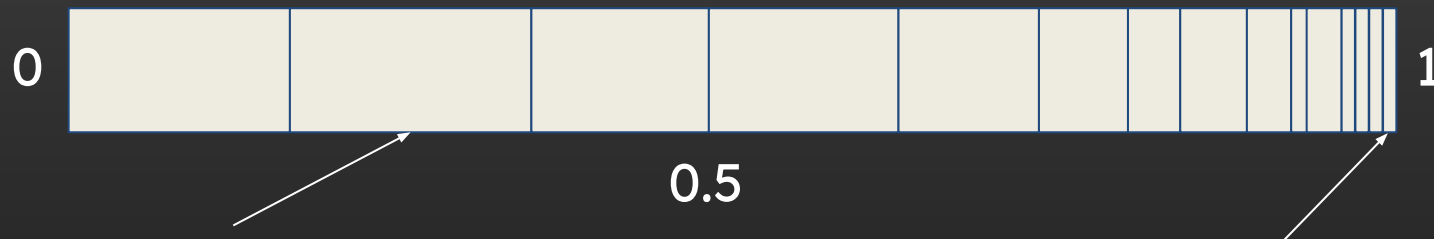
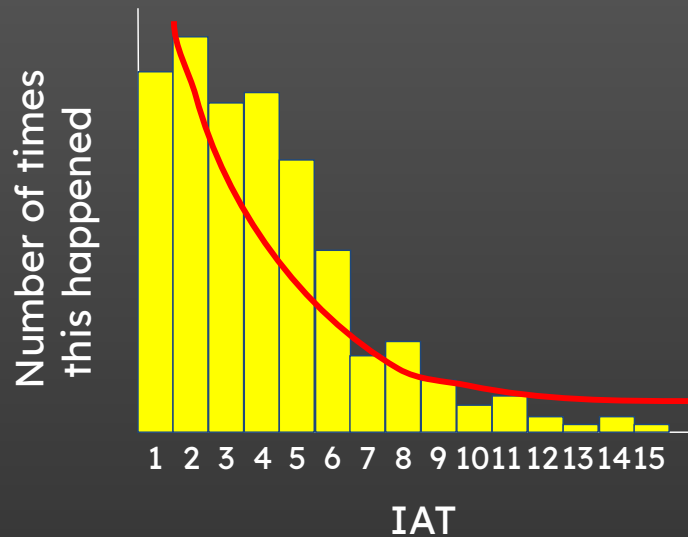
$$(0.4 \times 7 + 0.3) = 3.1 \rightarrow \underline{0.1}$$

Random Seeds



HSMA

For sampling from a distribution, the randomly generated number is then compared against a kind of scale, where (depending on the distribution) bigger parts of that scale equate to picking certain numbers.



Any random numbers in this range will emulate sampling a 2 minute IAT

A 15 minute IAT will only be sampled if the random number falls in this range

Random Seeds



HSMA

That's all well and good, but how do we get that starting number for our random number sequence?

Two ways :

- By default, the computer uses something unpredictable to determine the starting number (often the current date / time, milliseconds since a given date / time, IDs of processes running in the Operating System etc)
- Or, we can specify a *random seed*. In other words, we tell the computer what number from which to start its random number generation (this is often 42. Geeks know why). If we use a random seed, we'll always get the same random numbers.



Random Seeds



HSMA

Why use a random seed?

Reproducibility and **consistency**

Others can reproduce the exact results we got - good open science

Students can reproduce the exact results to make sure they've learned correctly

When we undertake “what if?” analysis, we can compare like with like - the only things that change are those that we want to change (not things like how many people came through the door). This improves the interpretability of the results.

We can more easily test our models when we know what the results should be

Exercise



HSMA

Split into your groups for 40 minutes + 15 minute break (5 mins now, then 10 mins at the end). As a group, you need to study the information about Dan's fish and chip shop ("Chalk's Chips") here :

<https://docs.google.com/document/d/1wDNHIEll3HCEj6T0tzErRONbSBJqmw9q2SEJGxrPn3w/edit?usp=sharing>

You need to draw up a diagram for what a DES model of Chalk's Chips could look like. In particular, you should clearly indicate :

- The generators and sinks, what they represent, and which entities the generators are bringing into being
- The activities to be modelled, what resources are needed (if any) for each activity, and what the associated queue represents in terms of the resources it's waiting for (if applicable)

If you manage to do all that in the time, consider if there are any attributes that the entities might have that might be useful to model.

When we come back we'll ask a few groups to present their designs. There are no right or wrong answers here - we do have a "Potential Solution" but model design is an art form, and there may be different ways you come up with to model the system **(IMPORTANT - you would also normally design a model based on the "what if?" questions it needs to answer, which you don't know here)**

In this next section, we're going to show you the Python code of a Discrete Event Simulation built using the Python Package *SimPy*.

We don't expect you to fully understand the code as we're not expecting you to be Python coders.

Instead, we'll talk you through the components of the SimPy model and then get you to work out what each bit of the code is doing.

Before we get to that though, there's a couple of concepts to which we need to introduce you that may be unfamiliar if you're not a Python coder, and which SimPy uses extensively.

Generator Functions



HSMA

SimPy is built around something known in Python as a *Generator Function*.

Conventional functions are structures in Python (and other programming languages) that allow us to specify how common things we want to do can be done, so we don't have to write the code from scratch each time.

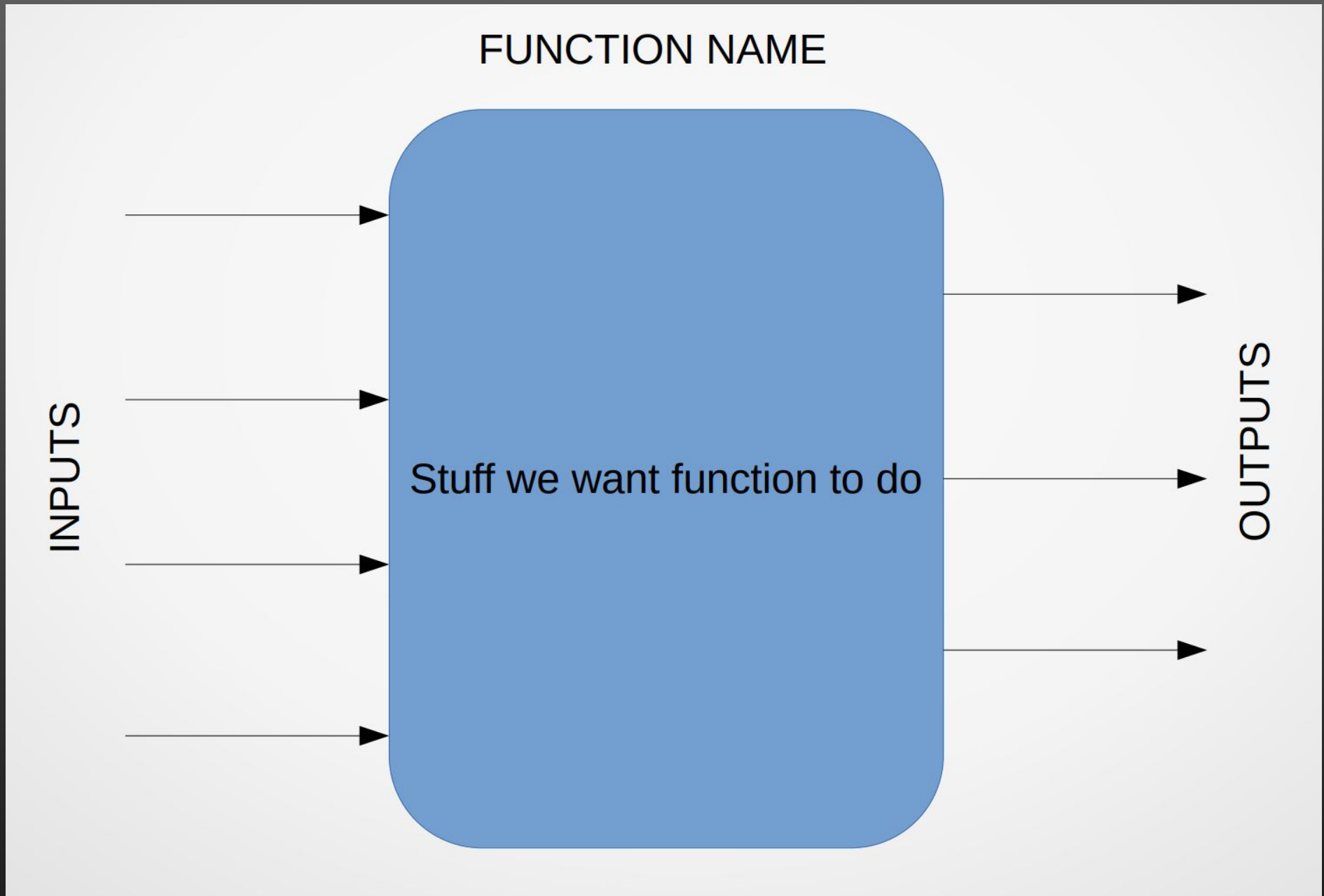
A function can take *inputs* (optionally), do something with them, and then return an *output* (optionally). The things it does are just sets of instructions.

A function is defined by giving it a name, the list of inputs it should expect (if any), the things we want it to do, and the outputs it should return (if any). The function doesn't do anything until it is called, however.

Generator Functions



HSMA



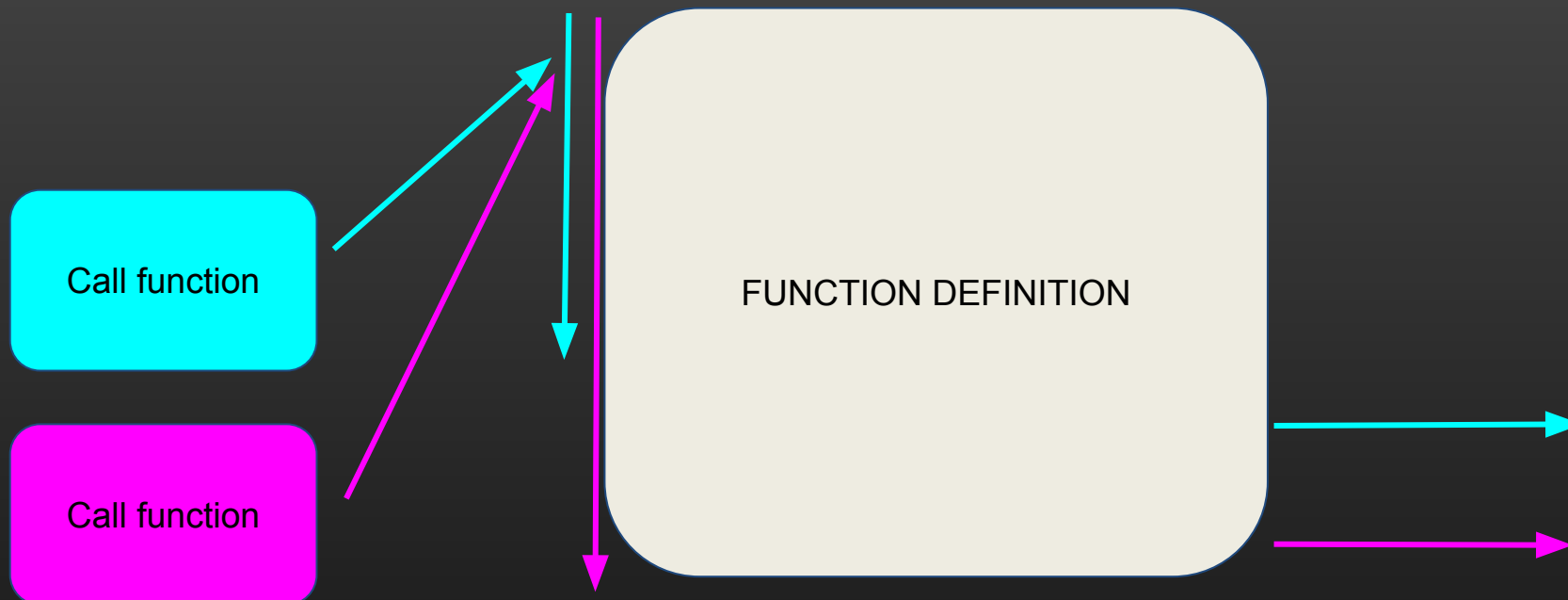
Generator Functions



HSMA

Conventional functions in Python are called, then run with some (optional) inputs, and then finish (usually by returning some output). When we call the function again, it runs again, from scratch.

In other words, conventional functions always start from the beginning every time they are called.

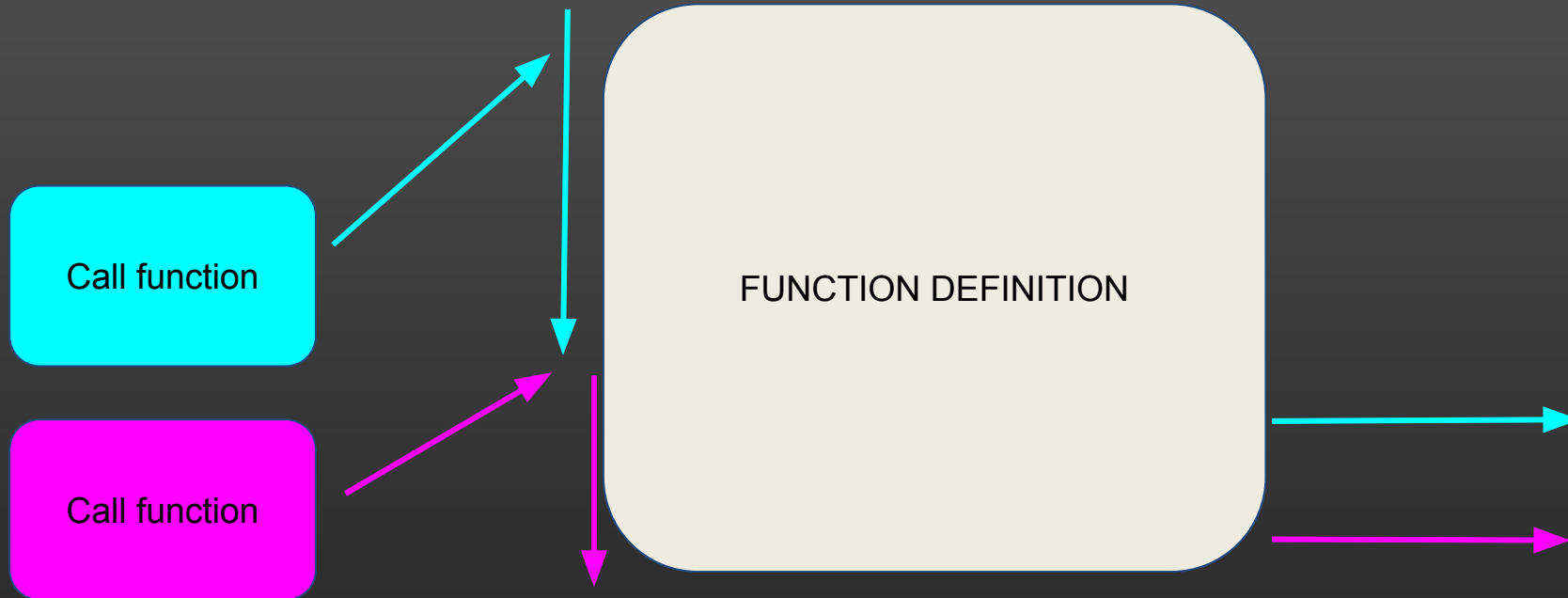


Generator Functions



HSMA

Generator functions remember where they were and what they did when control is passed back (they retain their *local state*), so that they can continue where they left off. We create an *instance* of a generator function (a copy of it) so we can have lots running all at different points.



This is very useful where we want state to be maintained. For example, we could remember how long until we generate the next entity, or where an entity is in a pathway...

Generator Functions in SimPy



HSMA

In SimPy, we use Generator Functions in two different places :

1. To model the DES generators (arrival points)
2. To model the individual journey of each entity

Let's imagine we are modelling patients in a patient pathway.

For 1, the generator function basically creates a patient, sets them off on their pathway, then freezes in place for an amount of time representing the inter-arrival time to the next patient. Then it does it all over again.

For 2, the generator function requests a resource and freezes until that resource is available (the queue). When the resource is available it freezes in place for an amount of time with it (the activity). It will then either move on to the next activity (and request the resource for it, as above) or end if there are no further activities.

Object Oriented Programming



HSMA

When you first learn programming, you typically initially learn the *procedural programming* methodology. This is where we write a program purely as a list of instructions to be carried out, one after the other.

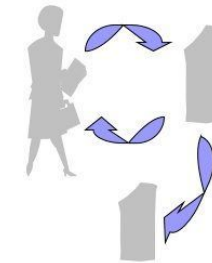
But an important concept in modern programming is *Object Oriented Programming (OOP)*.

In OOP, everything is centred around *objects*, properties they have and what they can do.

Let's look at an example.

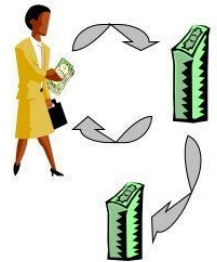
Procedural vs. Object-Oriented

■ Procedural



Withdraw, deposit, transfer

■ Object Oriented



Customer, money, account

Object Oriented Programming

Let's say we want to write a model that has lots of patients in it.

We don't want to have to define every patient using a separate piece of code.

Instead, we can write a definition of what a generalised patient *is* and *does* - a “blueprint” for a patient. And then whenever we need a new patient, we can say :

- We'll have a new one of those patient things I told you about
- And this one has these properties



HSMA



I'd like a new one of these.

Apparently...

Object Oriented Programming



HSMA

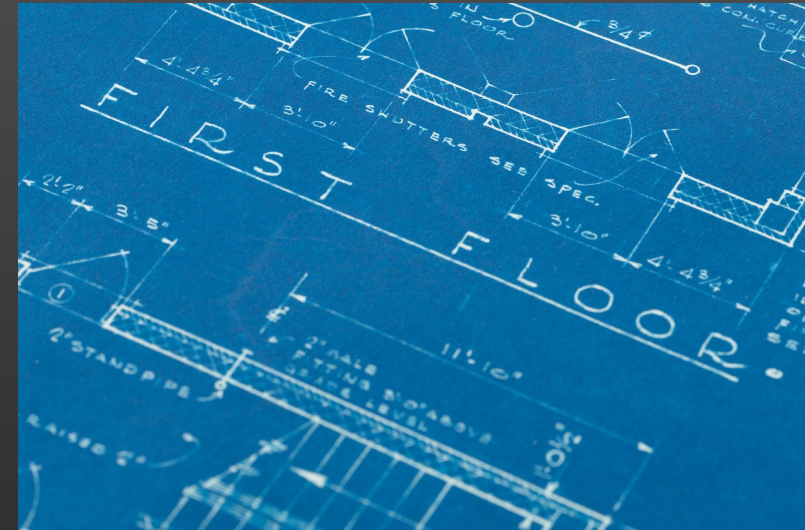
So how do we construct this “blueprint”?

Well, we can consider that a patient is made up of :

- Properties that the patient *has* - information about them (name, age, their condition, how they’ve been treated etc)
- Things that patients can *do* (call surgery, collect prescription, visit hospital)

In Object Oriented Programming, these blueprints are known as *Classes*, each of which have *attributes* (properties) and *methods* (things they can do).

Attributes are variables that store information, and *methods* are functions that do things.



Object Oriented Programming



HSMA

CLASS : Patient

Attributes

patient_id : int
priority : int
previous_admission_60_days : boolean

Methods

progress_condition (probability : float)
choose_ed_to_attend (options : list)



A nice way to think about attributes - imagine a patient carrying a clipboard with their information on it along with them on the pathway.

Sometimes that information is just read.

Sometimes that information is updated (eg maybe in this example, if the progress_condition method leads to a worsening, their priority might change)

Instantiation



HSMA

Our *class* acts as a blueprint for what a patient looks like. When we want to create one, we use a process known as *Instantiation* to create an *object* from the class definition.

Each object, whilst sharing the common attributes and methods of the class, is its own separate entity, with its own attribute values, and calling its own methods.

CLASS : Patient

Attributes

patient_id : int
priority : int
previous_admission_60_days : boolean

Methods

progress_condition (probability : float)
choose_ed_to_attend (options : list)

Object : patient_dan

Attributes

patient_id = 165
priority = 3
previous_admission_60_days = True

Methods

progress_condition (probability = 0.7)
choose_ed_to_attend ([Plymouth, Truro])

Object : patient_sammi

Attributes

patient_id = 321
priority = 1
previous_admission_60_days = False

Methods

progress_condition (probability = 0.4)
choose_ed_to_attend ([Exeter, Torbay])

Instantiation
Instantiation

Constructors



HSMA

A *constructor* defines what happens when an object is *instantiated* from a class (ie created from a blueprint).

The constructor essentially “constructs” the object, specifies the initial values for the attributes, and may even run some methods to start things off.

The constructor is itself a method within the class.

In our example, the constructor might set up the patient ID, randomly allocate a priority and previous admission value, and run the `choose_ed_to_attend` method to select where the patient arrives.



Me, Myself and Self



HSMA

self is an important concept in Object Oriented Coding in Python. Basically, *self* stores the instance of the class (ie the copy from the blueprint). It can be a bit weird to get your head around, particularly if you're new to coding.

We declare *self* as the first input to any method definition in the class. It basically says “Run this class method on *this instance*” (eg on the patient Dan).

Whenever we need to refer to something belonging to this instance, we say *self*. to say “Give me the x that belongs to you”. Eg give me the attribute value that you (the patient Dan) is carrying. Or run the method that belongs to you.

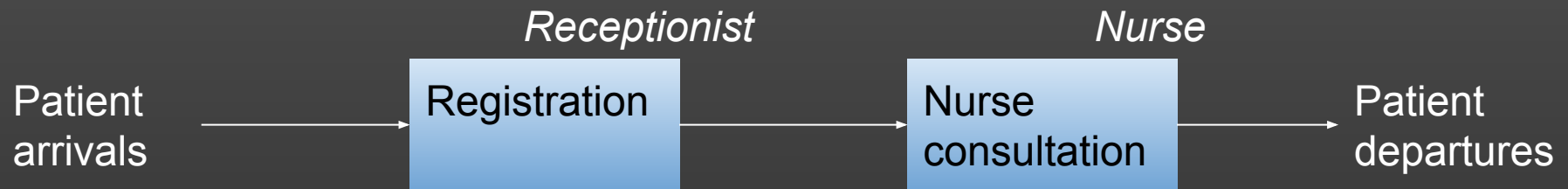
Basically, by using *self* we refer to the *instance* of the class (the individual *object*) rather than the *Class* itself (the blueprint).

Simple Model



HSMA

We're going to look at how a SimPy model is put together. In the example we're going to look at, we'll model a very simple system - patients arriving at a clinic for a nurse consultation. Patients register at reception, then see the nurse. One type of entity, one generator, two activities, two queues, one sink, two types of resource.



A SimPy model can seem quite complex at first, particularly for such a simple model as this. But the good news is the overall structure is always the same, regardless of complexity.

Entity Classes



HSMA

When building a SimPy model, we create a *class* for every type of entity that we want to model. In many models, there will only be one type of entity but the process is the same if we need others.

In our simple example, we have just one type of entity - patients.

We're going to have our patients represented quite simply here :

- We'll give them two attributes :
 - a patient ID (so we can identify each patient uniquely) - we'll give them this when we create them
 - records of how long they spent waiting for the receptionist and nurse, which we can update once they've finished queuing
- We won't give them any methods (other than the constructor)

CLASS : Patient

Attributes

id : int

q_time_registration : float

q_time_nurse : float

Methods

Simulation Time



HSMA

Before we go on, let's talk about how time is handled in our DES models. SimPy simulations run in *time units**. These units of time can represent any real world amount of time we like **as long as we are consistent within the same model** (e.g if we specify our inter-arrival times in minutes, everything else will need to be in minutes).

The good news is that SimPy allows for floating point time units. In other words, if we specify our activity times in minutes, but then need to specify that arrivals are coming in on average every 30 seconds, we can do that, by specifying half a time unit.

This is because SimPy doesn't run in time units ticking away one by one. Instead, it schedules events and jumps to the next event. But don't worry about that for your purposes. Just know that, because of this, even if you specify only integer average times, you will see current simulation time as floating point numbers (eg the current time unit could be 3.6 because it sampled a 0.6 activity time etc).

Param Class



HSMA

As well as our entities, we also need a class that will specify the *parameters* of our model. We'll call this class *Param*, and it'll give us a single place where we can change our parameter values (eg if we want to change the number of receptionists or nurses)

This will include things like how many resources we've got, what the average activity times are, how many replications of the model we want to run, how long to run it for etc.

We'll set up a class that has some attributes to store our parameters - we don't need any methods for this one beyond the constructor. We'll set it up so that we can pass in updated parameter values, but if nothing is passed in, we'll use default values instead.

Model Class



HSMA

We'll need to create a class for our simple model - but this one is much bigger. We need a class that defines the model itself and all its logic.

Specifically, this will hold attributes that will store :

- A copy of our parameter values
- Store something known as the SimPy environment (where everything lives)
- Store the SimPy Resource objects we need
- Store copies of the distributions we need (with random seeds, so we can replicate the “random” results)
- Keep track of any model run-specific information, such as within-run counters, results from this run and which replication we're running

Model Class



HSMA

We'll also have a number of methods in the Model class that will define the logic of the model :

- A method for each DES generator (arrival point) that will create new entities (patients) at a given rate
- A method for the pathway that our entities will be sent through (we may have more than one if there are separate pathways in the same model)
- A method to run the model
- A method to pull out the information from all the patients in a model run (replication) : think of this like collating the patients' clipboards at the end
- A method to calculate the results across the run

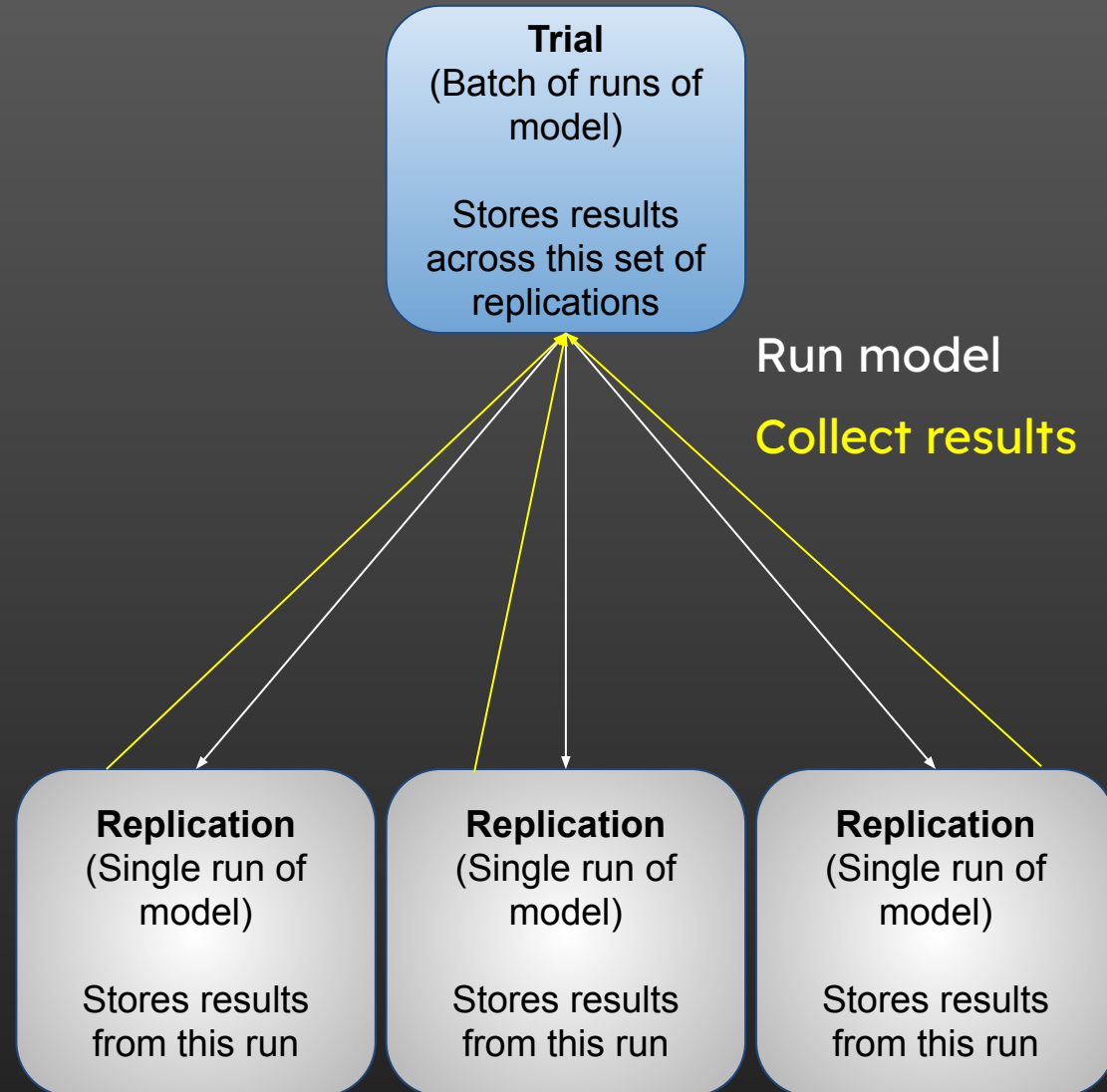
Trial Class



HSMA

The final class we need is the *Trial* class. This class will run a batch of replications (a trial) with a given set of parameter values, and will calculate average results over this batch of replications.

This way, every time we want to try out a scenario, we can create a new instance of a Trial with that set of parameter values.



Trial Class



HSMA

In the Trial class, we'll have attributes that store :

- A copy of the parameter values (that we can pass over to the Model class)
- Trial-level results once we calculate them

And we'll have methods that do the following :

- Run a batch of replications
- Calculate the results over the trial

Model Structure Summary



HSMA

Entity Class(es)

A class for each type of entity in our model. Each entity we create carries information around with them about themselves and how long they waited

Model Class

A class containing the logic of the model itself, and representing a single replication (run) of the model. Has methods to create entities, fire them off down pathways, run the model, extract information from the entities and calculate results over the run

Param Class

A class that stores the set of our parameter values that we want to test in our model

Trial Class

A class that runs a bunch of replications and calculates average results over that batch

Model Structure Summary



HSMA

PARAM CLASS

Stores the set of parameter values for the experiment



I SET THE RULES!

You run the experiment.

TRIAL CLASS

Runs many replications (Model runs) and calculates the average results

LET'S RUN
A BUNCH OF
MODELS!



MODEL 1



MODEL 2



MODEL 3



...

MODEL N



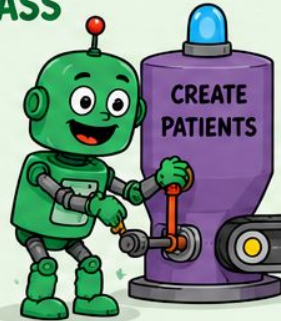
TRIAL RESULTS

Average
wait times,
utilisation,
throughput,
...and more!

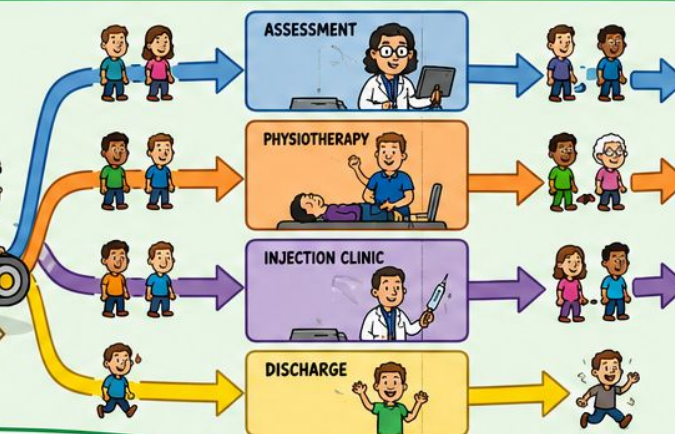


MODEL CLASS

Creates entities (patients) and sends them down pathways. Collects information and calculates results for the run



ONTO THEIR PATHWAYS! →



MODEL RESULTS

- Wait times
- Utilisation
- Throughput
- Other metrics



Param Class
Trial Class
Model Class

= Model (replication)
 = Patient (entity)

Exercise



HSMA

You will now split into your groups for 1 hour. You have been provided with a notebook on *Google CoLab*, which contains both markdown cells (cells containing narrative text etc), and code cells (that can be run using the play button next to them).

The notebook is here :

<https://colab.research.google.com/drive/1oXfZvHvxcUFfJIzw51UdIxoijSHOBP0o?usp=sharing>

Take a copy of the notebook by clicking File > Save a Copy in Drive. You will then work on your copy.

Exercise



HSMA

As a group, you should undertake the following :

Part A

Undertake Part A of the Notebook. You have been presented with code for a simple SimPy model, split across cells representing blocks of code from the model. Your task is to study each block of code and, in the markdown cell above, write a description of what you think the block of code is doing.

We're not expecting you to fully understand everything, but rather try to get the gist of what's happening based on the structure of a SimPy model we've just outlined and the concepts of generator functions and Object Oriented Programming.

To edit a markdown block, double click in it. When you've finished editing it, hit Shift + Enter to store the changes.

You should also run each code cell as you study them. Be aware that most of them won't appear to do anything as most of them are simply defining classes and functions. You should see a green tick next to the code block when it's finished running successfully. Make sure you run the first code block before you do anything else - this installs and imports the packages in the remote session to which you're connected. You do not need to write an explanation of this cell.

Exercise



HSMA

Part B

Undertake Part B of the Notebook. There are two tasks in Part B :

1. You are given instructions to write an additional “what if?” scenario, to run it and print the results. Using your interpretation of the code from Part A, work out what code you need to write in this section. Then run the cell and observe the results.
2. You are asked to change the model to incorporate an additional part of the pathway - 40% of patients branching off to see a doctor for a consultation after seeing the nurse. Read the instructions and tips carefully. You have been provided with the code blocks to which you need to make additions, along with comments indicating where you need to add code (or, in one case, make a change to an existing line of code). After you’ve made the changes to each cell, run it and verify it’s working. Assuming you’ve written everything correctly, the final code cell (once completed) should run the model correctly and produce outputs for the new doctor consultation activity.

Sammi and I will float around the rooms. Work as a team to undertake this exercise.

When we return, we’ll talk through the solution.

Animating Models with Vidigi



Link to slides:

hsma-programme.github.io/nhs_oa_vidigi_slides/

Link to activity:

hsma-programme.github.io/nhs_oa_vidigi_streamlit/

Resources: HSMA 6 Recordings



2B: SimPy for Discrete Event Simulation - Part 1

3 hours

SESSION LEAD: DR DANIEL CHALK

In this session, we look at how we can use the SimPy Python package (which is built around generator functions) to build Discrete Event Simulations in Python.

We start with a very simple model in which patients arrive, are seen by a nurse, and then leave, before building up to multi-step models and models with branching pathways. We then explore how to record results from our simulation and explore the impact of changing the level of resources available or the distribution of inter-arrival times and activity durations.

This content is also covered in part 2 of the HSMA little book of DES.

This session makes use of the [des](#) environment

📄 Slides

📺 Video

📁 Repository

📄 Open in Github
Codespaces

📖 Book

📺 Exercise Solution Video
- Part 1

📺 Exercise Solution Video
- Part 2

2C: SimPy for Discrete Event Simulation - Part 2

3 hours

SESSION LEAD: DR DANIEL CHALK

In this session, we look at some more advanced DES concepts and how we can incorporate them in our SimPy models.

The full recordings from HSMA 6's DES module can be found at hsma.co.uk/des

All of the other HMSA 6 content can be found at hsma.co.uk/modules

In this session, we've taught our new, improved approach to DES model structure - but these recordings and exercises still provide a great starting point.

Resources: Little Book of DES



HSMA



Licencing, Attribution and Acknowledgements

Part 1 - DES Concepts and Design

- 1 Introduction to DES Concepts
- 2 Exercise - Designing a DES

Part 2 - Your First SimPy Model

- 3 An Introduction to SimPy
- 4 The Recommended Structure for DES Models
- 5 An Example SimPy Model
- 6 Adding Multiple Activities
- 7 Adding Branching Paths
- 8 Exercise - Building Your First Model

Part 3 - Extending Your Basic Model

- 9 Warm Up Periods
- 10 Priority-Based Queueing
- 11 Modelling Resource Unavailability
- 12 Using Different Distributions for Different Processes

```
# Simulation meta parameters
sim_duration = 2880
warm_up_period = 1440 ##NEW - this will be in addition to the sim_duration
number_of_runs = 100
```

Tip

If you find it easier to keep track of, you could define your warm-up like this instead.

```
results_collection_period = 2880
warm_up_period = 1440
total_sim_duration = results_collection_period + warm_up_period
```

9.3.2 The patient class

Our patient class is unchanged.

9.3.3 The model class

In the model class, the 'attend_clinic' method changes.

We look at the current elapsed simulation time with the attribute `self.env.now`

Then, whenever a patient attends the clinic and is using a nurse resource, we check whether the current simulation time is later than the number of time units we've set as our warm-up.

9.3.3.1 The attend_clinic method

```
# Generator function representing pathway for patients attending the
# clinic.
def attend_clinic(self, patient):
    # Nurse consultation activity
    start_q_nurse = self.env.now

    with self.nurse.request() as req:
        yield req
```

Table of contents

- 9.1 How long should a warm-up period be?
- 9.2 Implementing the warm-up period
- 9.3 Coding the model
 - 9.3.1 The g class
 - 9.3.2 The patient class
 - 9.3.3 The model class
 - 9.3.4 The trial class
- 9.4 The impact of the warm-up period

Our free eBook at des.hsma.co.uk covers all content from the HSMA 6 DES module, as well as significantly expanding it with more advanced concepts.

Again, this uses the older approach to writing HSMA models, but it still provides a great resource for this stuff!

Resources: DES RAP Book



HSMA

DES RAP

Step-by-step guide

Examples

Frameworks ▾

Intros ▾

Impact

GitHub

Choose your language:

Python

R

Welcome to our step-by-step guide to DES RAP

PAGE LAST MODIFIED
February 11, 2026

Introduction

Setup ▾

Version control

Environments

Structuring as a package

Code organisation

Conceptual modelling

Model inputs ▾

Input modelling

Input data management

Parameters from script

Parameters from file

Parameter validation

Model building ▾

Randomness

Entity generation

Entity processing

Logging

Output analysis ▾

How to use this guide

Work through each page **in order**. Follow along with the code examples as you go. You'll gradually build up a working model and your skills as you progress.

Each section builds on the previous one. You can:

- Follow the explanations and instructions
- Copy and run the provided code
- Complete exercises at the end to test your understanding

It will help you meet two important reproducibility standards:

NHS Levels of RAP

STARS reproducibility recommendations

On this page

[How to use this guide](#)

[Before you get started...](#)

[Example models](#)

[Getting help](#)

[Reporting problems](#)

Produced by our ARC colleague Amy Heather, this book is a thorough guide to reproducible DES modelling in both Python and R.

pythonhealthdatascience.github.io/des_rap_book

Resources: Atlas



A screenshot of the 'DES projects' page on the Atlas website. The page has a teal header with navigation links: 'Welcome', 'Packages, Projects and Tools', 'Techniques', 'Graph Catalogue', 'About', 'Contribute', and icons for GitHub, a plus sign, and a search icon. On the right side, there is a 'Contents' menu with links for 'Software', 'Training', and 'DES projects' (which is highlighted). The main content area is titled 'DES projects' and includes a sub-header 'Below are some projects making use of DES from across the health service.' It displays three project cards. The first card is 'ADHD CYP Pathway DES' with a 'STATUS: BRONZE' badge, showing a line graph and a brief description of DES modelling for ADHD. The second card is 'AmbModelOpen: A Discrete Event Simulation Of Ambulance Resources' with a 'STATUS: GOLD' badge, showing a bar chart and a description of a Discrete Event Simulation in R. The third card is 'DES RAP: Stroke Capacity Planning Model' with a 'STATUS: GOLD' badge, showing a flowchart and a description of two equivalent repositories for stroke capacity planning. Each card has buttons for 'Details', 'Code', and 'Web app' or 'Documentation'. At the bottom, there are more project cards, including one with a grid of small line graphs and another with a complex flowchart.

The Healthcare Services Analytics & Decision Support Atlas collates projects from the NHS and related organisations.

The DES page collates a range of resources and models:

atlas.hsma.co.uk/techniques/des/

Resources: Vidigi Package



HSMA

[vidigi](#) [Getting Started](#) [Customising Animations](#) [Feature Breakdowns](#) [API Reference](#) [Example Models](#) [Migrating to 2.0.0](#) [Changelog](#) [About](#) ▾



Examples Gallery

Explore what's possible with vidigi through these hands-on examples.

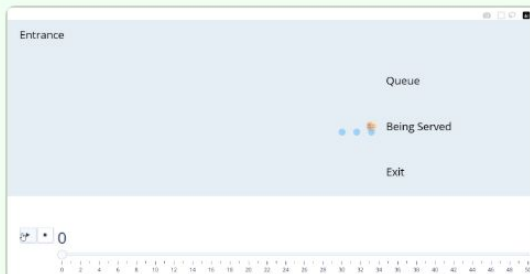
SimPy Examples

A collection of models built using the SimPy framework.



A very simple example with one server

The simplest possible starting point - a single-step SimPy model with just one server. This example walks through using vidigi's EventLogger helper class to capture arrivals, queueing and resource use with the minimum of code changes to an existing model.



A slightly more complex example with multiple servers

Building on the single-server example, this model introduces multiple servers, so we need a way to track which specific resource each entity is using. Here we bring in vidigi's VidigiStore class, a drop-in replacement for SimPy's Resource

On this page

[SimPy Examples](#)

[Ciw Examples](#)

[Salabim Examples](#)

[Agent Based Simulation \(ABS\) Examples](#)

[Advanced Examples](#)

[Legacy Examples](#)

If you are looking to add vidigi to your own models, check out the documentation to find a 'getting started' guide, extensive examples, and more.

hsma-tools.github.io/vidigi/

ReModel Club: Read, Run, Reuse Open Simulation Models



The ReModel club is a new club run by the strategy unit, focussing on **Running** and **Reusing** healthcare simulation models.

Running ~monthly for 1 hour, their goals are to “empower people within the NHS to:

- run open source simulation models on their own machines
- critique/assess/evaluate simulation models with their pros and cons
- repurpose existing simulation models for their own use cases”

Sign up here:

the-strategy-unit.github.io/ReModel-docs/join-us/#register-for-remodel

The sessions are free to attend, but spaces will be capped at 15 places per session on a first-come, first-served basis; NHS analysts will receive priority.

Sign up for HSMA 7



We are hoping to launch HSMA 7 in late Spring/early Summer 2027.

HSMA 7 will feature our biggest rewrite yet, with the DES module expanding from 9 hours to **over 30 hours of content**, covering DES modelling in Python in more detail than ever before.

Find out more by visiting [**hsma.co.uk/hsma7**](https://hsma.co.uk/hsma7)

Want to hear when applications open?

Sign up to our mailing list by emailing
[**penchord@exeter.ac.uk**](mailto:penchord@exeter.ac.uk)

Follow us on LinkedIn at
[**linkedin.com/company/hsma-programme/**](https://linkedin.com/company/hsma-programme/)

What else will you learn on HSMA 7?



In HSMA 7, we are doubling down on our two most impactful areas.

“Where is demand, and where should I put my services?”

Geographic Modelling

In this module, students learn key geographic principles and how to create high-quality maps.

They are also introduced to advanced operational research techniques for solving problems around where to place services in the face of competing priorities like equity of access, costs, and net zero goals.



“I want to identify patients to target with interventions”

Machine Learning

In this module, students learn the key principles of supervised machine learning, allowing them to train machine learning models designed to support operational improvements to services through better targeting of interventions.

They are also introduced to issues around model bias, ethics, legal issues, and model explainability.



What else will you learn on HSMA 7?



Web Application Development

Students will learn to use the Streamlit app framework, allowing them to create engaging interactive web apps to bring powerful techniques like simulation into the hands of non-coders



Interactive Reporting

Students will learn Quarto, a publishing system that allows them to build parameterised data-driven static and interactive reports



Data Visualisation

Students will learn to generate engaging static and interactive charts and maps in Python with industry-standard packages



Automation

Students can combine these techniques to automate and improve regular reporting

How is HSMA Structured?



PHASE 1: Learn

6 months training, 1 day a week



PHASE 2: Apply

9 months on a real project for your organisation

1 day a week expectation, with guidance and support



PHASE 3: Implement

From month 15 onwards, receive support for implementation and impact gathering



Need More Convincing?



HSMA

“My HSMA journey has been a transformative experience, opening me up to new approaches to analytics in the NHS.”

“HSMA is an absolutely amazing programme that’s taken me from zero coding knowledge to inspiring me to want to go off and explore on my own.”

“I can honestly say it is the best training I’ve ever participated in. For the first time in a long time I feel really enthused about my work.”



H S M A

FIND THE FUTURE IN YOU

Exit Poll



HSMA



menti.com/alquykryaozi

Your responses will be used to help us evaluate and improve our training. We may also use anonymised responses for research and evaluation purposes, including in academic publications, conference presentations, reports and other research outputs. Participation is voluntary.